A Practitioner-Oriented Review of Reinforcement Learning for the Joint Optimization of Ads and Organic Content in Large-Scale Recommender Systems

ARMANDO ORDORICA, University of Toronto, Canada
YURI LAWRYSHYN, University of Toronto, Canada

Balancing advertisements with organic content is a challenge for large-scale recommendation platforms. Ads are central to platform revenue, but jointly serving them with organic content creates trade-offs between monetization and user experience. Many production systems frame these decisions as supervised learning problems that optimize immediate ad clickthrough, estimating immediate outcomes under a historical logging policy while treating decisions as independent. Formulating joint ad and organic ranking as long-horizon optimization over action sequences, under policy-induced state evolution, remains an open research problem.

Reinforcement Learning (RL) offers a principled framework for modeling recommendation as sequential decision-making under uncertainty, but at scale it must represent high-dimensional state and action spaces, specify robust reward functions, explore safely, and manage substantial computational cost. This practitioner-oriented survey synthesizes prior RL work on the joint optimization of ads and organic content, distilling design choices, trade-offs, and evaluation practices through a unifying taxonomy that organizes each system by how it formulates the Markov Decision Process (MDP) in terms of state, action, and reward, and learns its policy. We find that industrial systems favor hybrid architectures and offline or constrained RL, such as contextual bandits and counterfactual policy evaluation, that learn from logged data while balancing scalability, safety, and interpretability.

Additional Key Words and Phrases: Reinforcement Learning, Ad Policy Optimization, Contextual Bandits, Recommendation Systems, Off-Policy Learning, Online Advertising, Deep Reinforcement Learning, Multi-Objective Optimization

1 Introduction

Large-scale content recommendation platforms in industry deploy systems whose revenue models depend on jointly optimizing monetization and long-term engagement in interactive environments [103]. Failing to achieve an adequate balance can result in issues such as ad fatigue, where users become less responsive to ads due to overexposure, and user churn, where users discontinue using the platform [67, 74].

Jointly optimizing revenue and long-term engagement is structurally difficult for several fundamental reasons. First, the monetization and engagement objectives can conflict: actions that increase immediate monetization, such as higher ad load, typically degrade future engagement and retention [67, 94]. Second, immediate per-interaction signals are directly observable and easier to optimize, whereas long-term outcomes unfold over extended horizons and are harder to measure and attribute to specific actions [37, 55, 85]. Third, current decisions shape the user responses used to update future decisions, making the optimization signal endogenous to the policy, i.e., shaped by the very decisions it is meant to evaluate [40, 109]. These properties make the trade-off inherently sequential and policy-dependent.

To address these challenges, large-scale platforms have relied on supervised machine learning models optimized for short-term engagement signals such as clicks, dwell time, and conversions [20, 21, 56, 96]. By learning rich representations of users, items, and context from large-scale interaction logs, these models estimate conditional expectations of immediate outcomes and use these predictions to guide content selection and ranking. This paradigm has delivered substantial improvements in short-term engagement [20] and click-through prediction performance [56]. It remains a backbone of large-scale industrial recommender systems [59].

Authors' Contact Information: Armando Ordorica, University of Toronto, Toronto, Canada; Yuri Lawryshyn, University of Toronto, Toronto, Canada.

2025. Manuscript submitted to ACM

While neural networks remain powerful function approximators, optimizing supervised objectives on historical logged data does not directly address the sequential and interactive nature of the system. In such settings, supervised learning (SL) is structurally insufficient for two key reasons. First, SL ignores policy-induced distribution shifts. It optimizes conditional expectations of immediate outcomes (e.g., $\mathbb{E}[r_t \mid s_t, a_t]$) using data generated under a historical policy. This implicitly assumes that the training distribution is fixed. In interactive systems, however, the recommendation policy shapes user behavior, and user behavior in turn determines future training data. This violates the stationarity assumptions underlying standard supervised optimization [25, 78]. Second, SL lacks a principled mechanism for evaluating counterfactual trajectories as it relies exclusively on realized trajectories. Because user journeys are highly personalized and path-dependent, most possible interaction sequences are never observed in logged data. Long-term outcomes therefore depend not only on the current state-action pair but also on the policy that generates the subsequent trajectory [6].

Reinforcement learning (RL) provides a framework in recommendation systems to solve the two problems above [2]. First, RL explicitly represents policy-induced distribution shifts by modeling recommendation as a Markov Decision Process (MDP) in which actions influence future states, providing the framework within which RL algorithms can account for this dependence [77]. Second, RL leverages off-policy estimators that re-weight observed trajectories under a candidate policy to evaluate counterfactual trajectories from logged data, and enables policy improvement by optimizing the expected cumulative return under policy-induced dynamics [78]. Rather than replacing SL, RL can be viewed as embedding supervised prediction within a sequential control framework [19]. Deep models remain central for representation learning and reward estimation, while RL augments them with the machinery required to optimize policy-dependent, long-term objectives [26, 49].

Despite the promising theoretical framework that RL offers for optimizing long-term value in recommendation systems, implementing RL in real-world, large-scale platforms remains complex [26]. First, the state space, which reflects the current context of the user and the environment, and the action space, which captures how the recommender system can be tuned, are both essentially infinite [108]. This vastness makes learning and optimization computationally intensive and requires scalable approximation methods, as exact dynamic programming over such state and action spaces is infeasible in practice [2]. Second, designing appropriate reward functions is non-trivial, as poorly chosen reward functions can lead to unintended system behaviors that degrade user experience or negatively affect business revenue [29]. For example, a reward function that overly prioritizes click-based metrics may drive the system to promote clickbait content, boosting short-term engagement but eroding user trust and satisfaction over time [89]. Conversely, a reward function too heavily skewed toward revenue metrics could result in excessive ad load, degrading the user experience and accelerating churn [68]. Furthermore, interpretability is a major challenge in RL deployment. Understanding the learned policies can be particularly difficult when policies are updated in real-time during user interactions (online policy learning) [26, 35, 84]. In extreme cases, this lack of transparency could result in non-compliant or unethical behavior, posing significant risks to both users and the business [48].

In light of the challenges outlined above, this paper provides a synthesis of RL approaches to monetization-aware recommendation systems, using the MDP elements (state, action, reward) and policy learning as a unifying taxonomy. Although previous surveys address RL in recommendation systems [2, 51], our systematic literature search (see Section 2) did not identify any survey that explicitly focuses on the joint optimization of ads and organic content in sequential and interactive environments, which leaves limited guidance for systems that must jointly reason about monetization and long-term user value. This paper addresses this limitation by reviewing approaches that move beyond supervised

objectives defined under fixed logging policies to analyze how RL agents model and optimize sequential trade-offs between revenue and engagement.

This survey introduces a component-level taxonomy that clarifies how existing systems operationalize state, action, reward, and policy, and uses it to systematically organize and analyze more than 150 academic and industrial works. Beyond the categorization itself, we synthesize the design choices, trade-offs, and evaluation practices that recur across these systems, with particular attention to how each balances the joint optimization of monetization and long-term engagement. The resulting framework is intended as a practical reference for industry practitioners exploring how RL components have been combined with supervised learning in production-scale ad and recommendation systems.

The paper begins with a historical overview of recommendation systems with monetization components, tracing the evolution from rule-based and heuristic approaches to tree-based models and modern deep learning architectures. This progression highlights the limitations of short-horizon optimization and motivates reinforcement learning as a framework for modeling long-horizon, policy-dependent objectives. The remainder of the paper is structured around these components (state, action, reward, and policy) using them as analytical categories to compare and contrast existing systems. Rather than structuring prior work chronologically or by algorithmic family, we compare systems according to how they define and operationalize these components.

2 Paper Collection Methodology

To ensure broad coverage, relevant papers were collected from both academic and industry sources. Foundational academic works such as *Reinforcement Learning: An Introduction* [77], *Statistical Methods for Recommender Systems* [6], and *Artificial Intelligence: A Modern Approach* [65] were used to establish key mathematical foundations, including Markov Decision Processes (MDPs), Contextual Bandits, Off-Policy Learning, and Offline Replay.

To reflect the internal perspective of a production-facing ad policy team, 14 papers from an industry curated reading list were also compiled. Examples include *Jointly Learning to Recommend and Advertise* [103], *DEAR: Deep Reinforcement Learning for Online Advertising Impressions in Recommender Systems* [101], *Ad-load Balancing via Off-Policy Learning* [68], and *Practical Bandits* [81]. In addition, targeted keyword searches such as "Reinforcement Learning ads [company name]" and "Contextual Bandit [company name]" were conducted for leading companies in the advertising and recommendation space (e.g., Google, Meta, Pinterest, Microsoft, TikTok, YouTube, LinkedIn), surfacing additional papers describing real-world systems and methodologies. To complement those efforts, a separate search was conducted on Google Scholar for relevant review papers. While no surveys were found that specifically focused on RL in ad recommendation systems, related reviews on "contextual bandits", "ad fatigue", and "recommender systems" were used to identify additional references.

Finally, manual crawling of citation networks in cornerstone papers was used to identify additional key contributions. For instance, tracing citations from *Jointly Learning to Recommend and Advertise* [103], *DEAR* [101], and *Offline Reinforcement Learning* [49] led to works such as *Ads Allocation in Feed via Constrained Optimization* [94] and *A Contextual Bandit Bake-off* [10]. In total, 153 papers were selected and reviewed. Papers were ranked according to three main criteria: influence, to capture the foundational or widely adopted ideas in the field; industry relevance, to ensure that the outcomes were sound, implementable, and actionable; and recency, to reflect the latest methodological developments and deployment practices.

While recent and highly cited work was prioritized, papers introducing novel methodologies were also included regardless of citation count. To contextualize the selected works, this survey reviews the five design questions that structure the RL formulation: reward design, action-space design, state-space representation, policy learning, and

RL construct	Notation	Role in monetization-aware recommendation
Environment	$\mathcal{E}$; dynamics $P(s_{t+1} \mid s_t, a_t)$	The platform setting that reacts to the ranking policy: users, ad inventory, organic items, auction context, and the interface where ads displace or accompany organic content.
Goal	$\max_\pi \mathbb{E}_\pi[\sum_{t=0}^{T} \gamma^t R_t]$	Learn a policy that optimizes a weighted utility over monetization, engagement, retention, and user-experience costs. The weights encode the platform's chosen trade-off among these objectives.
Episode / horizon	$\tau = (s_0, a_0, r_0, \ldots, s_T); T$	A user session, repeated-visit window, or A/B experiment horizon over which cumulative effects of ad and organic ranking decisions are measured.
State	$s_t \in \mathcal{S}$	The partially observed user, session, request, context, ad-history, and organic-interaction signals available before a ranking decision.
Action	$a_t \in \mathcal{A}^{\text{ad}} \cup \mathcal{A}^{\text{org}}$	The decision the platform takes at a ranking opportunity: which ads are eligible, how many ads to show, where to place them, which quality thresholds to enforce, and how to order organic items.
Reward	R_t or $R(s_t, a_t)$	The scalar utility that combines monetization and user-side outcomes, such as revenue, clicks, conversions, dwell time, retention, and ad-fatigue penalties.
Policy	$\pi(a \mid s)$ or π_θ	The learned rule that maps observed context to ranking actions, thereby controlling ad load, placement, eligibility, and organic ordering.
Exploration strategy	e.g., ϵ, Upper Confidence Bound (UCB), Thompson sampling	The action-selection strategy that allocates traffic to uncertain ad or organic ranking decisions to reduce uncertainty about outcomes not yet observed with enough support. These are examples rather than an exhaustive list.
Logging policy	$\mu(a \mid s)$	The historical or experimental policy that generated logged impressions used for offline replay, counterfactual evaluation, and off-policy learning.

Table 1. Notation guide for RL formulation in monetization-aware recommendation

exploration-exploitation. These design questions are not a one-to-one expansion of the MDP tuple; rather, they identify the modeling choices that recur when RL is applied to industrial recommender systems with ads. Drawing on existing methods and lessons from industry, the resulting framework is intended to inform practitioners exploring how RL can be applied in their organizations to develop ad policies that balance monetization with organic engagement. Table 1 serves as a compact notation guide for the recurring RL constructs used throughout the paper, including the environment, goal, episode, actions, and policy.

3 A Brief History of Advertising and Recommendation Systems: From Rule-Based to Deep Learning-Based Joint Optimization

The goal is not to provide a general history of online advertising, but to explain why earlier approaches became insufficient for monetization-aware feeds that must balance advertiser value, platform revenue, user engagement, and long-term retention. The history begins with rule-based advertising, which gave platforms simple control over eligibility and placement but could not personalize ads at scale. Performance-based pricing reframed ad allocation around observable outcomes such as clicks and conversions. Supervised machine learning and representation learning then improved platforms' ability to estimate conditional expectations from logged interaction data, but these methods remained largely pointwise and myopic. Bandits and offline policy evaluation introduced adaptive policy selection under uncertainty, but they still struggled when present ad decisions shaped future user states. RL therefore appears as the next step for modeling ad recommendation as a sequential policy-optimization problem. This evolution is summarized in Figure 1 and Table 2.

Table 2 summarizes the historical evolution of ad optimization using the MDP elements that organize the rest of the paper: reward design, action spaces, state spaces, and policy learning. These dimensions evolved somewhat in parallel rather than strictly sequentially, and modern systems typically combine advances from all of them. The table keeps the

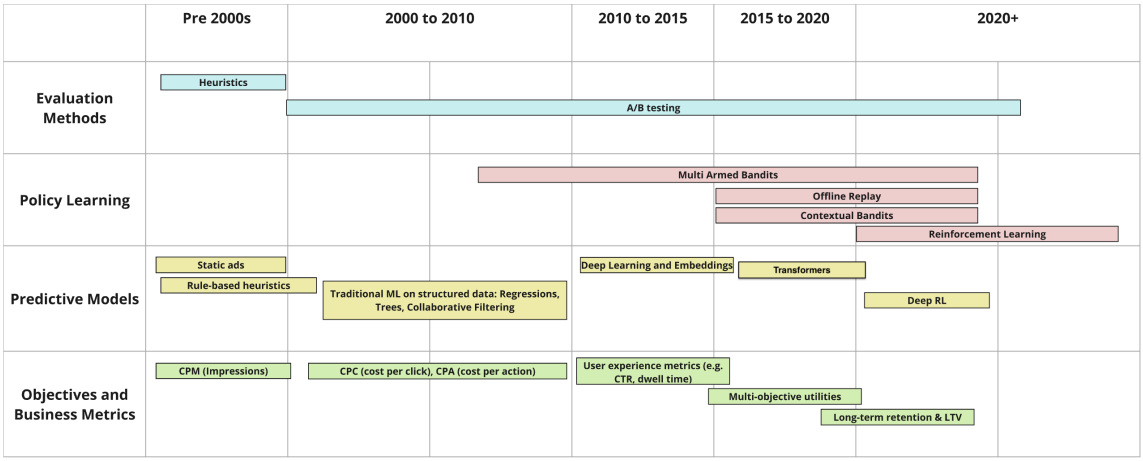

Fig. 1. Timeline showing initial adoption and popularization of techniques and methodologies in online advertising. The boundaries and lengths of the rectangles are approximate. Innovations and transitions between techniques and objectives occurred gradually and continuously, rather than sharply at specific dates.

history visible while previewing the design questions that recur in the RL formulation: what signal is optimized, what decisions the system can make, what user and context information the policy observes, and how candidate policies are learned and evaluated. Evaluating these choices is difficult because reward functions, action discretizations, state definitions, and policy-learning methods create a large and often intractable search space. Although online A/B testing remains the gold standard for causal validation, testing every candidate live is prohibitively expensive and risky. Offline replay and related counterfactual methods are therefore used to narrow the set of promising candidates before A/B testing, balancing broad coverage of the design space with high-confidence online validation.

3.1 From Rule-Based Advertising to Traditional Machine Learning

Around the early 2000s, online advertising largely mirrored offline practices, heavily relying on non-targeted formats such as banner ads [45]. These generic advertising methods conflicted with the objective of search engines of efficiently directing users to relevant content [30]. As a marginal improvement on nontargeted ad formats like banner ads, advertisers began to utilize simple `if-then` logic with predefined rules to select and display ads based on basic criteria like user demographics. While rule-based systems were straightforward to implement, their lack of flexibility and personalization became evident as ad fatigue, defined as reduced user engagement due to repeated exposure to the same advertisement, persisted [1]. This issue underscored the necessity of not only leveraging user signals but also optimizing ad load tuning parameters, such as frequency and placement, to maintain user interest and maximize click-through rates [1]. For instance, researchers at LinkedIn introduced concepts like top slot - highest eligible position for an ad within the feed - and minimum gap - the required distance between consecutive ads to prevent oversaturation - as ad load tuning parameters [94]. Their research indicated that users are more likely to click on ads that are spaced farther apart [94].

The evolution of ad pricing, from paying for impressions to paying for user actions, created pressure within the online advertising industry to improve relevance and performance-based metrics [38]. Sponsored search, introduced by GoTo

Reward Design			
Stage	**Optimization Signal**	**Limitation**	**When Sufficient**
Cost per Mille (CPM)	Impressions and reach	Weak alignment with relevance or user value	Brand exposure and guaranteed reach
Cost per Click (CPC)	Clicks and traffic	Can overvalue immediate engagement	Direct-response campaigns
Cost per Action (CPA)	Conversions and purchases	Sparse feedback and attribution challenges	Clear conversion funnels
Utility functions	Revenue, engagement, fatigue, and other objectives	Objective weights are hard to choose and validate	Explicit multi-objective trade-offs
Representative sources: [28, 30, 38, 94].			
Action and State Spaces			
Stage	**Action / State Representation**	**Limitation**	**When Sufficient**
Rules and segments	Eligible ads, placements, demographics, and location	Coarse action sets and limited personalization	Small inventories or strict policy constraints
Tabular features	Engineered user, ad, item, and session variables	Limited nonlinear interaction modeling	Interpretable models or limited compute
Embeddings	Dense user, ad, item, and context vectors	Pointwise similarity does not model policy feedback	Large-scale retrieval and ranking
Sequential encoders	User histories, evolving interests, and session context	More complex training and harder action attribution	Session-aware ranking and dynamic intent
Representative sources: [20, 21, 83, 105].			
Policy Learning and Evaluation			
Stage	**Horizon Modeled**	**Limitation**	**When Sufficient**
Rule-based	None; static allocation	No learning from user response	Stable, low-complexity policy settings
Supervised machine learning (ML)	Pointwise immediate response	Treats logged impressions as independent labels	Decisions are approximately independent
Bandits	One-step exploration and exploitation	Limited treatment of future state changes	Few policy arms and immediate rewards
Contextual bandits	One-step decisions with user and item context	Actions are not modeled as changing future states	Real-time adaptation with short-horizon goals
Full RL	Multi-step cumulative return	Harder reward design, evaluation, and deployment	Ad fatigue, retention, and long-horizon objectives
Representative sources: [10, 34, 50, 78, 101].			

Table 2. Historical Evolution of Ad Optimization Through the Main RL Design Dimensions. The table summarizes how earlier advertising and recommendation methods shaped reward signals, action and state representations, and policy-learning choices in modern monetization-aware recommendation systems.

(later Overture), allowed advertisers to bid on specific keywords, directly linking ads to user queries and addressing the limitations of generic advertising [30]. Foundational pricing models, such as Cost per Mille (CPM), charged advertisers per 1,000 impressions and closely mirrored offline practices [38]. Yahoo!, through the acquisition of Overture [76], later introduced Cost per Click (CPC) [27], a performance-based pricing model that shifted costs to reflect measurable user engagement. This evolution continued with the introduction of Cost per Action (CPA), which tied fees to specific user actions beyond clicks, such as purchases or sign-ups. These changes incentivized the advertising industry to innovate, leading to the development of more relevance-focused strategies. These foundational pricing models remain integral to the utility functions that underpin contemporary ranking algorithms [30]. As advertisers increasingly demanded performance-based pricing, the ad tech industry was compelled to innovate within a competitive yet lucrative market,

addressing the limitations of rule-based and heuristic models through the adoption of machine learning techniques to enable more dynamic and data-driven approaches [99].

The next set of innovations following heuristics and performance-based targeting was led by machine learning (ML) and A/B testing [9]. From 2010 to 2015, ad selection techniques evolved significantly with the adoption of machine learning models such as Logistic Regression, Decision Trees, and Gradient Boosting Machines [22]. By leveraging historical data and diverse user behavior signals, these models enabled more accurate predictions of user engagement [22]. In parallel, A/B testing allowed practitioners to systematically compare models and features, ensuring that any new implementation delivered measurable performance gains [9]. As ad and content inventories grew, representation learning became central to large-scale retrieval and ranking. Industrial recommender systems used deep models to encode users, items, and context for candidate generation and ranking [20, 21]. Two-tower architectures extended this pattern by learning separate user-side and ad-side representations that could be matched efficiently at scale [83]. These models improved the state representation available to ad-ranking systems, but they usually remained trained on pointwise or short-horizon objectives such as clicks, conversions, or engagement estimates. As a result, they did not by themselves resolve the reward-design and policy-learning challenges created by revenue-engagement trade-offs, ad fatigue, and long-term retention.

Multi-Armed Bandits (MABs) are a class of algorithms for dynamic decision-making under uncertainty designed to tackle the explore-exploit tradeoff: finding the optimal balance between choosing new ad policies with unknown performance and showing older ad policies with known positive performance [12, 72]. Their goal is to reduce unnecessary exposure to suboptimal treatments by continuously updating the estimates of expected reward and uncertainty as new data arrives [47]. In the context of online advertising, these "treatments" are typically specific ad configurations, defined by parameters such as displacement cost, reserve price, ad load, and ad placement, while "rewards" represent user engagement metrics such as clicks or conversions [72].

Although the theoretical foundations of this technique date back to early- and mid-20th-century probability theory [64], the computational infrastructure required to support live, large-scale implementation did not exist until much later. As a result, our literature search identified broader industrial adoption of MABs in web and advertising platforms only beginning in the late 2000s and early 2010s [17, 50, 72].

In contrast to traditional A/B tests, where there are randomized treatments assigned for a predetermined amount of time, MABs dynamically learn which configuration yields the highest reward for each user at a particular time. Although this adaptive learning framework manages the trade-off between minimizing regret from suboptimal treatments and exploring potentially superior configurations, defining the precise reward function to optimize remains an open challenge [86].

To formalize multi-objective trade-offs such as balancing revenue and user satisfaction, utility functions have been employed in online advertising since the early 2000s [28]. Drawing on microeconomic utility theory, platforms formalized multi-objective trade-offs through scalar utility functions that encode multiple objectives, such as long-term revenue and engagement, into a single metric, enabling principled comparison and optimization [28]. When integrated into MAB frameworks, this unified reward signal guides exploration toward policies that balance short-term performance (e.g., clicks or conversions) with long-term considerations such as user trust and fatigue.

Although utility-weighted bandits can encode long-term targets via proxy rewards, they remain structurally one-step: they estimate conditional expectations under a logging policy and do not represent policy-induced feedback loops, thereby limiting counterfactual evaluation of multi-step trajectories under alternative policies.

3.2 Advances in Deep Learning: Attention Models, Transformers, and Contextual Bandits

This section explains how deep learning, sequential models, and contextual bandits advanced the goal of jointly optimizing engagement and revenue over time before the adoption of full reinforcement learning. The discussion is organized into three parts: first, it establishes why interleaving ads within organic content streams created a joint optimization problem; second, it examines how neural representations improved pointwise response prediction and how Transformer architectures scaled sequential user modeling through attention; and third, it discusses how contextual bandits enabled adaptive exploration while remaining constrained by short-horizon objectives.

As platforms evolved from static web pages with dedicated ad slots to unified, infinite content feeds, the fundamental ad-selection problem changed in two distinct stages. First, in early web layouts, advertisements were confined to isolated placements outside the primary content area, such as banner regions or dedicated sidebars in search results, which allowed platforms to optimize ad selection independently from organic content [99]. Second, as feed-based architectures emerged, platforms introduced fixed slotting within the feed itself, inserting ads at deterministic positions, such as every N-th post or pin. Although fixed slotting placed ads inside the organic stream, the choice of which ad to insert at a given slot remained largely decoupled from the organic ranker. However, in modern feeds, ads and organic items occupy the same set of positions, so every ad insertion displaces a candidate organic item. The cost of that displacement depends primarily on the relative relevance of the ad and the displaced organic item to the user's current intent. When commercial intent is high, for example a user comparing products or planning a purchase, an ad with a relevant outbound link can serve the user better than the next organic item, lowering or even reversing the displacement cost; when intent is low, the same ad insertion sacrifices a more engaging organic alternative. Although platforms can still rank ads and organic items in isolation, doing so ignores how this relevance-driven displacement cost varies across requests and systematically leaves revenue-engagement trade-offs unmanaged. Joint ad and organic ranking addresses this by explicitly trading off immediate ad revenue against the engagement cost of displacing more relevant organic content [94, 103].

To support this joint optimization, deep learning improved the representation of users, ads, organic items, and their interactions. In particular, learned embeddings advanced representation by encoding users, ads, organic items, and context in a shared feature space, while neural ranking models improved the mapping from those representations to immediate objectives such as click and conversion signals [20, 21, 83]. While these representational advances successfully improved short-term response prediction, they fundamentally relied on the assumption that impressions could be treated as independent events. Because the underlying models were trained on pointwise labels from logged impressions, they optimized for immediate outcomes under a historical logging policy rather than evaluating the sequential impact of ad insertion on future user states [19, 78]. Consequently, these models weakly captured how the order, spacing, and accumulation of prior recommendations shaped later outcomes. This representational gap motivated sequential models, including Transformer-based architectures, which can encode user histories and changing intent from ordered interaction sequences [44, 82, 105]. Even with better sequence representations, however, the core policy-learning problem remains unresolved: the platform must still choose actions, such as ad load, placement, reserve prices, or ranking policy, whose effects are only observed through delayed user responses and future logged data [68, 109].

To address this policy-learning gap, *Offline Replay* is often employed to find optimal policies in large parameter search spaces [6, 7, 49]. This technique typically begins with data collection via A/B/n experiments, where different policies are randomly assigned to users [4, 18]. Afterward, Offline Replay simulates the effects of various ad policies across different user segments and estimates the potential rewards based on logged interactions [6]. In this context,

"reward" refers to the performance metric being maximized, such as clicks, likes, shares, or revenue. However, in many cases, the true optimal treatment is not explicitly tested [4]. Here, importance sampling corrects for differences between the observed behavior policy and the target policy, improving estimation accuracy [25, 77]. Based on the collected data on random assignments between policies and users typically involved in Offline Replay, one can use different policy learning methods to map the optimal policies from user ids, content types, and other contextual signals for the optimal weights at any point in time [78].

Offline replay often assumes that user preferences and item availability do not change significantly during data collection. However, in real ad systems, these factors can shift, causing offline data to become stale or biased [3, 49]. Contextual bandits address this limitation by leveraging continuous feedback from live user interactions, preventing the policy from being locked into a static offline dataset [50]. Contextual bandits can be thought of as a special case of RL, where an action selection does not influence future states [11, 77]. Unlike full RL, which models long-term state transitions, contextual bandits assume that each decision is independent, optimizing for immediate rewards rather than a multi-step trajectory [11, 77]. This makes them particularly well-suited for recommender systems, where ranking decisions can be treated as independent events [2, 107]. Additionally, contextual bandits naturally balance exploration and exploitation, allowing them to adapt dynamically to user behavior in real time [50]. However, a practical limitation of contextual bandits is that they select actions from a fixed set defined at deployment time, such as preconfigured ad policies or treatment arms [10]. As a result, the policy's performance is inherently constrained by the quality and granularity of the available states and actions at the time of inference [24].

Although contextual bandits inherently focus on short-term rewards, they can be tuned to approximate long-term objectives by encoding historical or delayed outcomes into the feature space or the immediate reward function [55, 89]. For instance, delayed retention or repeat-visit signals can be folded into proxy rewards so that the policy favors configurations associated with longer-term user value [95]. However, full RL remains more appropriate for scenarios demanding explicit multi-step optimization, where actions taken now significantly shape future user states [34]. Contextual bandits thus offer a practical trade-off, providing continuous adaptation and efficient online learning without the computational complexity of fully modeling multi-step state transitions [34, 49].

Despite the practical advantages that contextual bandits offer for real-time experimentation and short-horizon reward maximization, they remain limited in scenarios where user states evolve over repeated interactions [77]. Modern advertising platforms often contend with extended user sessions and multiple recurring visits, demanding an approach capable of modeling multi-step feedback loops. RL naturally extends these capabilities by incorporating stateful dynamics and cumulative rewards, capturing phenomena like ad fatigue [15] and long-term retention [89]. Recent industrial deployments underscore RL's growing importance in ads: ByteDance uses RL-based systems [101, 103] to balance immediate revenue with session-level engagement in short-video feeds, while Meta (Facebook) has leveraged policy-gradient methods for sequencing notifications and ad placements over evolving user states [34]. These efforts demonstrate how RL can transcend the myopic focus of bandits to address long-horizon outcomes, multi-objective trade-offs, and dynamic user behaviors. In the sections that follow, we explore how these RL paradigms are integrated into large-scale advertising stacks, focusing on core algorithms, design decisions, and lessons learned from real-world implementations.

4 The RL Formulation of Jointly Optimizing Ads and Content

4.1 Formulating Joint Ad and Organic Ranking as a Markov Decision Process

We formalize joint ad and organic content ranking as a Markov Decision Process [63, 77], defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. At each timestep t, the platform observes a user state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, transitions to a new state s_{t+1} according to the dynamics $P(s_{t+1} \mid s_t, a_t)$, and receives a scalar reward $R_t \in \mathbb{R}$. The discount factor $\gamma \in [0, 1]$ trades off immediate against delayed feedback. The policy $\pi(a \mid s)$ governs which action the platform takes in each state, and the objective is to find $\pi^\star = \arg\max_\pi \mathbb{E}_\pi\left[\sum_{t=0}^{T} \gamma^t R_t\right]$. Table 1 summarizes how each of these components is operationalized in industrial ad systems.

The action space decomposes into ad and organic candidates, $\mathcal{A} = \mathcal{A}^{\text{ad}} \cup \mathcal{A}^{\text{org}}$, with the policy choosing both what to insert and where, rather than treating ads and organic items as independent ranking problems [68, 94]. The reward R_t similarly aggregates ad-side and organic-side outcomes through a utility function (Section 4.2), so that the policy evaluates monetization and engagement within the same optimization objective rather than optimizing each signal in isolation.

Two practical complications separate this formulation from the fully observed, on-policy MDP abstraction used to introduce the notation above. First, the platform observes behavioral traces (clicks, dwell time, profile attributes) rather than the user's latent intent or satisfaction, so the state representation is inherently *partially observable* [42, 53]. Second, in an on-policy setting, the same policy generates the data used to update or evaluate it. By contrast, ad systems often learn from historical impressions collected under an already-deployed ranking or allocation rule, called the *logging policy* $\mu(a \mid s)$. A new candidate policy π_θ may choose different ad loads, placements, or organic items from the same state, so the logged data do not directly show what would have happened under that candidate policy. Offline learning and evaluation therefore must account for the gap between μ and π_θ through importance weighting or doubly robust estimation [25, 49, 78].

The remainder of Section 4 is organized around the components of this MDP. Section 4.2 discusses how R is specified to balance monetization and engagement. Section 4.3 addresses the structure of $\mathcal{A}$, including the ad-versus-organic split and constraints such as ad load and frequency capping. Section 4.4 reviews state representations that summarize partially observed user histories. Section 4.5 surveys methods for learning π under the off-policy data conditions noted above. Section 4.6 treats exploration as the deliberate divergence between μ and π_θ used to discover better policies under uncertainty.

4.2 Reward Design for Monetization-Aware Recommendation Systems

This subsection provides a structured synthesis of reward design in monetization-aware recommendation systems. It first discusses the properties a reward function should ideally satisfy and why reward specification is difficult in practice due to partial observability, sparse and delayed long-term outcomes, and competing objectives such as revenue, engagement, and ad fatigue. It then reviews how prior work operationalizes these objectives through proxy metrics and blended utility formulations, along with the rationale for these choices and their main limitations.

Relatively simple reward functions may be sufficient in controlled settings, where the objective is directly specified and outcomes are immediately observable. Reward design is substantially harder in monetization-aware recommendation systems, where the platform must infer long-term user value from noisy behavioral traces while balancing monetization against user experience.

In principle, an effective reward signal for monetization-aware recommendation systems should satisfy at least three properties. First, it should be sufficiently informative about the user's latent preferences and intent, even though these quantities are not directly observed. Second, it should provide feedback at a timescale and frequency that makes learning feasible, even when the platform ultimately cares about sparse and delayed long-term outcomes such as retention or habit formation. Third, it should faithfully encode trade-offs among competing objectives, such as revenue, engagement, and ad fatigue, rather than overemphasize any single short-term metric in isolation.

In practice, however, satisfying these properties is difficult for at least three reasons. First, reward design is constrained by *partial observability*: the system must infer users' latent preferences and intent through a fixed set of observable signals that capture different stages of response, such as immediate reactions (clicks), deeper consumption (dwell time or completion), and downstream value signals (saves, purchases, or return visits) that provide only a noisy and incomplete view of the underlying user state. Second, many of the outcomes platforms ultimately care about are *sparse and delayed*: retention and longer-term satisfaction may take weeks or months to manifest, and the effect of any single intervention is often subtle, making meaningful changes difficult to detect within the short time horizons of typical experiments [55]. Third, reward specification is inherently *multi-objective*: recommendation engines must simultaneously account for potentially competing goals such as revenue, engagement, and ad fatigue, rather than optimize a single scalar outcome in isolation [68, 94, 99].

Metrics such as click-through rate (CTR), predicted CTR, and ad impressions are commonly used as proxies for monetization, while session duration, saves, shares, and return-based signals stand in for longer-term engagement [85]. Their appeal is obvious: they provide dense, behaviorally grounded feedback that is relatively easy to log, predict, and optimize. Yet this same convenience is also their main limitation. When optimized in isolation, such proxies can misrepresent the platform's true long-term objective [54]. Click-based signals provide a clear example: although they are frequent and immediately observable, high CTR does not necessarily imply high ad quality, and repeated exposure to low-quality or poorly matched ads can degrade user satisfaction and future responsiveness to ads [37, 75, 94]. Even within the same family of signals, semantics can vary: short clicks may function as implicit negative feedback, whereas longer clicks or post-click dwell may better reflect genuine value. Similar tensions arise elsewhere. Diversity-related signals, for instance, may be desirable when they reflect broader use-case coverage over time, which prior work associates with improved long-term user experience and retention [85, 92], yet high click entropy within a session can also indicate confusion, low relevance, or friction [67]. Tables 3, 4, and 5 summarize representative proxies for revenue, engagement, and ad fatigue. Overall, the issue is not the absence of informative signals, but the fact that each captures only a narrow and context-dependent slice of user value. This is precisely what motivates composite utility functions that blend multiple signals rather than relying on any single proxy in isolation.

While proxy metrics ensure that the system has informative observable signals for multiple latent aspects of user behavior, the utility function determines how those signals are combined into a single optimization objective. Some desirable properties of utility functions overlap with those of proxy metrics: the objective should remain aligned with long-term platform value, provide reward signals dense enough to support learning, and capture the relevant trade-offs among competing goals. However, utility functions also introduce an additional requirement that is distinct from proxy design: structural correctness of aggregation. In particular, the utility must specify not only which components enter the objective, but also the direction of each component's contribution, its relative magnitude, and the functional form through which components interact. This includes whether effects are linear or nonlinear, additive or interaction-dependent, and whether they exhibit saturation, thresholds, or state dependence. In monetization-aware recommendation, this

distinction is critical because engagement, revenue, and fatigue signals rarely combine in a purely additive or context-invariant way. As a result, even well-chosen proxies may yield a misspecified objective if they are combined with incorrect signs, poorly calibrated coefficients, or an inappropriate functional form. For example, ad revenue may increase utility up to a point but reduce it beyond a saturation threshold; dwell time may be informative only within a certain range; and an ad click followed by a hide may convey a different signal than either event in isolation.

Proxy Metric	Short Description	Polarity
Click-through rate (CTR)	Approximates revenue potential; higher CTR often correlates with higher ad clicks [94, 99, 103].	**Positive** (higher is better)
Predicted CTR (pCTR)	Model-estimated probability that a user will click on an ad.	**Positive** (higher is better)
Predicted Revenue	Often a function of bids and pCTR to estimate expected revenue [94, 99].	**Positive** (higher is better)
Click Yield	Ratio of total clicks (ads + organic) to total impressions on a page; helps avoid optimizing CTR in isolation [99].	**Positive** (higher is better)
Ad Impressions	Number of times an ad is displayed to users.	**Mixed** (context-dependent)

Table 3. Revenue proxy metrics and their polarity.

Proxy Metric	Short Description	Polarity
Session Duration	Tracks how long users stay engaged per session (proxy for longer-term engagement).	**Positive** (higher is better)
30-day Total Sessions	Aggregated count of user sessions over a 30-day window, proxy for habit or stickiness [92].	**Positive** (higher is better)
Likes / Adds to Library	High-intent actions indicating deeper user satisfaction and long-term retention [55].	**Positive** (higher is better)
Shares	High-intent action reflecting strong engagement.	**Positive** (higher is better)
High-quality Consumption	Consumption of content with high engagement (e.g., full video views, long reads) [85].	**Positive** (higher is better)
User Return Rate (URR)	Fraction of users who revisit the platform after a fixed interval [89].	**Positive** (higher is better)
Relative Watch Time	Percentage of a video watched, normalized by video length [90].	**Positive** (higher is better)
Dwell Time	Actual time spent viewing/reading an item, capturing depth of engagement [96].	**Mixed** (threshold-dependent: < 5s negative; ≥ 10s positive)
Ratio-based Diversity	Proportion of unique topic clusters in consumption history [85].	**Mixed** (context-dependent)
Repeated Consumption	How often users revisit similar content, reflecting satisfaction [85].	**Mixed** (context-dependent)
Distribution-based Diversity	Entropy-based spread of consumed topics [85].	**Mixed** (context-dependent)
Short-view Rate (SVR)	Fraction of impressions where watch time is below a low threshold (e.g., < 5s) [61].	**Negative** (higher is worse)
Early-skip Rate	Fraction of videos skipped or swiped away within the first few seconds (e.g., < 2 to 3s) [61].	**Negative** (higher is worse)
Skip / Not-click Rate	Fraction of impressions with explicit negative feedback signals (e.g., skips or non-clicks), indicating low relevance [102].	**Negative** (higher is worse)
Non-completion Rate	Fraction of plays that end before a minimum completion threshold (e.g., $< 20\%$ watched) [61].	**Negative** (higher is worse)
Time to Revisit	Interval between consecutive visits; shorter intervals suggest higher retention [85].	**Negative** (higher is worse)

Table 4. Engagement proxy metrics and their polarity.

Proxy Metric	Short Description	Polarity
Depth	Reduction in engagement (e.g., fewer impressions per session) over time [67].	**Negative** (higher is worse)
Length	Decrease in time spent per session [67].	**Negative** (higher is worse)
Frequency	Decrease in number of sessions per week [67].	**Negative** (higher is worse)
Ad Load (Sessions)	Number of sessions that end (or quit) during an ad [67].	**Negative** (higher is worse)
Ad Load (Absolute)	Absolute ad volume across sessions and changes in that volume [67].	**Negative** (higher is worse)
Click Entropy	Uncertainty/diversity in ad clicks for a query; high values can indicate lower relevance [67].	**Negative** (higher is worse)

Table 5. Ad fatigue proxy metrics and their polarity.

4.2.1 Why Blending Ad and Organic Reward Signals Is Structurally Hard. Combining ad and organic signals into a single utility is harder than combining signals within either content type alone. Two structural asymmetries between the two content types make this particularly challenging. First, they operate on different reward timescales. Monetization outcomes such as clicks and conversions are typically attributed within a short, well-defined window and therefore generate relatively dense, observable feedback. Conversely, organic engagement outcomes such as retention, habit formation, and long-term satisfaction manifest over much longer horizons, often measured across weeks or months [37, 55]. A simple additive utility collapses this temporal distinction into a single scalar, implicitly treating a same-day ad click as equivalent to a longer-term retention signal.

Second, the two objectives are expressed in incommensurable units. Ad utility is typically computed as bid $\times$ pCTR, a monetary quantity [94, 99], whereas organic utility is expressed in behavioral units such as saves, shares, or dwell time [55, 85]. Combining them requires a conversion factor that translates engagement into revenue-equivalent units. The appropriate value of this factor is context-dependent: it can shift across user segments, content types, and business priorities [37], and different platforms operationalize it differently, through constrained optimization [94], context-sensitive valuation [14], or separate reward functions with learned weighting [103].

A canonical utility function at the policy layer to blend ads and organic content can be represented as follows [14, 68, 94]:

$$U(x) = \alpha \times \text{Revenue} + \beta \times \text{Engagement} - \gamma \times \text{Ad Fatigue}$$

where α, β, and γ are weights that determine the relative importance of each factor. To compute the utilities of "Revenue", "Engagement", and "Ad Fatigue", typically many signals feed into each component. These signals include, but are not limited to, the ones shown in Tables 3, 4, and 5. In a bandit or RL context, $U(x)$ serves as the reward signal guiding policy updates.

4.2.2 Utility Function Validation and Reward Design. Validating and improving a utility function involves two distinct problems. Reward validation verifies whether a given configuration of weights, signs, and functional form produces the intended outcomes, and whether the specified utility function $U(x)$ tracks the platform objectives it is meant to represent. In practice, this is assessed through online A/B experiments that track top-line platform metrics such as daily, weekly, and monthly active users (DAU/WAU/MAU) for engagement and direct revenue for monetization [34, 37]. Standard A/B tests are typically run long enough to account for day-of-week variation and reach statistical significance (commonly on the order of one to two weeks in practice) and are well-suited for detecting short-term changes in click-through rate, revenue, and session engagement [37]. However, certain effects manifest over longer periods and are not reliably detectable within such short windows [37]. To capture these delayed effects, platforms employ long-term holdout groups, in which a subset of users is withheld from a new policy for an extended period (e.g., several months),

allowing the cumulative effect to be measured before a comparison is drawn [37, 55]. Iterative live validation of reward weights is also reported in the literature. Zhao et al. [103] describe a multi-objective reward framework at ByteDance in which separate reward functions are defined for organic content and ads, with the relative weights between them validated through a sequence of live experiments.

The second problem, reward design, is substantially harder: the goal is to identify a better-specified $U(x)$ by iterating over the reward function itself, that is, its component selection, coefficient magnitudes, signs, and functional form. The number of candidate configurations grows combinatorially with the number of parameters: each additional weight or proxy component multiplies the number of combinations that must be evaluated, making exhaustive A/B testing prohibitively expensive at scale [34]. This creates two compounding problems. First, the sample size required to detect meaningful differences across a large number of treatment arms grows with the number of comparisons, straining experimentation infrastructure. Second, A/B testing is inherently limited to the treatments that are explicitly tested [25, 78]; it provides no information about the performance of configurations that were never deployed. Offline replay addresses this second limitation by enabling counterfactual estimation: logged interaction data from randomized experiments is re-weighted using importance sampling to estimate how untested configurations would have performed under historical traffic [25, 49, 78]. This allows practitioners to evaluate a much larger set of candidate configurations cheaply before committing to a live experiment.

Offline RL extends this further by not only evaluating fixed configurations but actively searching for improved reward formulations within the space of possible configurations, using logged data as a surrogate for live interaction [49]. Rather than testing each configuration independently, offline RL methods learn a policy that maximizes the blended utility while staying close to the behavior policy that generated the data, thereby limiting the risk of out-of-distribution extrapolation [19, 49]. This makes offline RL a natural fit for reward design in large-scale systems, where the cost of live experimentation is high and the search space is too large for exhaustive enumeration.

Alibaba's whole-page optimization formulation illustrates a more constrained alternative to searching over many reward-weight configurations. Researchers at Alibaba introduced the concept of Click Yield, which is defined as the ratio of the total number of clicks on all items (ads + organic results) to the total number of impressions on a page [99]. Click Yield provides a holistic evaluation of page performance, accounting for the interactive effects between ads and organic content, and helping to mitigate the bias that arises from examining CTR in isolation. Their optimization task maximizes total revenue while ensuring that the Click Yield does not fall below a certain threshold (T) [99]. A comparison of utility function formulations is shown in Table 6.

Company	Paper	Organic Content Reward/Utility	Ads Reward/Utility	Blended Reward/Utility
TikTok	[101]	User Experience (r_t^{ex}): Measures the negative impact of displaying an ad on the user's experience, with a binary outcome. A positive reward (1) is given if the user continues browsing after seeing the ad, while a negative reward (-1) is given if the user leaves the platform after seeing the ad.	Ad Income (r_t^{ad}): Represents the immediate revenue generated from displaying an ad. It provides a positive value if an ad is displayed, contributing to maximizing advertising income.	$r_t(s_t, a_t) = r_t^{ad} + \alpha \cdot r_t^{ex}$ Here, α is a hyperparameter that controls the trade-off between maximizing ad revenue and minimizing negative user experience.
TikTok	[103]	$R_{\text{rec}}(s_t, a_t)$ Engagement metric (e.g., dwell time, clicks, purchases)	$R_{\text{ad}}(s_t, a_t) = \begin{cases} 1 & \text{if user stays after ad} \\ 0 & \text{if user leaves after ad} \end{cases}$	$R_{\text{total}}(s_t, a_t) = \alpha \cdot R_{\text{rec}}(s_t, a_t) + \beta \cdot R_{\text{ad}}(s_t, a_t)$
Alibaba	[99]	Click Yield (CY): Measures the overall performance of the entire page, considering both ads and organic content. It is defined as the ratio of total clicks on all items (both ads and organic) over the total number of impressions. $CY_{ij} = \frac{\sum_{n=1}^{N} Click(i,j,n)}{Imp(i,j)}$ Where $Click(i,j,n)$ is the click for item n in list j, and $Imp(i,j)$ is the number of impressions.	Revenue (REV): For sponsored content, revenue is computed based on the click-through rate (CTR) and cost per click (CPC). $REV_{ij} = \sum_{n=1}^{N} CTR(i,j,n) \times CPC(i,j,n)$ The CPC is determined by the auction mechanism (usually Generalized Second Price Auction).	The system optimizes the trade-off between revenue (REV) and Click Yield (CY). The optimization problem is defined as: $\max \sum_{i,j} REV_{ij} \cdot x_{ij}$ Subject to the constraint: $\frac{\sum_{i,j} CY_{ij} \cdot x_{ij}}{\sum_{i,j} x_{ij}} \geq T$ Where T is a threshold for the average Click Yield, ensuring user satisfaction.
LinkedIn	[94]	Expected Engagement Utility: Measures user actions like clicks, comments, shares, or time spent on organic content. The goal is to maximize engagement with organic content.	Expected Revenue Utility (for ads): Evaluates the revenue generated from ads, typically calculated as the product of the bid amount and the predicted click-through rate (pCTR).	The utility function for ads and organic content is formulated as: $\max \sum_i x_i r_i - \frac{w}{2}\|x\|^2$ Subject to the engagement constraint: $\sum_i x_i u_{ai} + (1 - x_i) u_{oi} \geq C$ Where: • x_i is a decision variable determining whether to place an ad in position i, • r_i represents the revenue utility for ad i, • u_{ai} and u_{oi} represent the engagement utilities for ads and organic content, respectively, • C is the engagement threshold. The shadow bid α acts as a conversion factor to blend the two objectives by equating engagement utility and revenue in a comparable manner.

Table 6. Comparison of Utility/Reward Formulation Approaches across Different Companies

Critical comparison of utility formulations. Table 6 presents four distinct utility formulations from three companies. While all four seek to balance ad revenue with user experience, they differ substantially in how they represent each

objective, how the two objectives are combined, and what signals are excluded. Understanding these differences is essential for practitioners selecting or adapting a formulation for their own systems.

Blending mechanism. Two of the four formulations use a linear weighted sum of separate ad and organic reward signals. In DEAR [101], the blended reward is $r_t = r_t^{ad} + \alpha \cdot r_t^{ex}$, where α is a scalar hyperparameter set manually. Similarly, Zhao et al. [103] use $R_{\text{total}} = \alpha \cdot R_{\text{rec}} + \beta \cdot R_{\text{ad}}$, with α and β tuned via live experiments. These additive forms are simple to implement and straightforward to interpret, but they implicitly assume that one unit of ad revenue is commensurable with one unit of engagement, an assumption that is context-dependent and difficult to validate without careful calibration [37]. In contrast, Alibaba [99] avoids direct blending altogether by formulating the problem as constrained optimization: revenue is maximized subject to a minimum Click Yield threshold T. This sidesteps the unit-conversion problem by treating engagement as a hard constraint rather than a term in the objective, but it introduces a different design choice: setting T requires understanding what level of Click Yield is acceptable, which must itself be validated. LinkedIn [94] takes a third approach, using a shadow bid α as an implicit conversion factor that translates engagement utility into revenue-equivalent units during the optimization, making the exchange rate explicit and learnable, though it still requires regularization via w to prevent the revenue term from dominating.

Representation of ad reward. The formulations differ considerably in how they define the ad-side reward. DEAR [101] and Zhao et al. [103] use binary signals: a reward of +1 if the user continues browsing after the ad and 0 or −1 if they leave. This is an intuitive proxy for user tolerance of an ad, but it is extremely coarse. It treats all ad exposures that do not immediately cause churn as equally valuable, regardless of whether the user clicked, converted, or simply ignored the ad. Importantly, it also does not capture revenue magnitude: a high-value ad that the user tolerates is treated identically to a low-value ad. Alibaba [99] and LinkedIn [94] instead define ad utility in monetary terms, as CTR × CPC and bid × pCTR respectively. These formulations are more aligned with the actual revenue objective of the platform, but they require accurate CTR or pCTR estimates, which introduce their own modeling challenges, particularly for rare or cold-start ads [56].

Representation of organic reward. The organic-side signals also vary in richness. Zhao et al. [103] define R_{rec} broadly as an engagement metric (dwell time, clicks, or purchases), leaving the specific operationalization unspecified. DEAR [101] frames the organic reward as a binary user-experience signal identical in structure to its ad reward, which again sacrifices nuance for simplicity. Alibaba's Click Yield [99] is a page-level aggregate that treats organic and ad clicks interchangeably in the numerator, which avoids the need to specify separate organic and ad utilities but loses the ability to distinguish between the two content types or to penalize ad fatigue. LinkedIn [94] explicitly separates engagement utilities for ads (u_{ai}) and organic content (u_{oi}) into a shared constraint, making the trade-off structure the most transparent of the four.

Limitations of existing utility formulations. A consistent limitation across all four formulations is the absence of explicit ad fatigue or long-term engagement signals. None of the blended utilities incorporate signals such as session exit rates due to ads, long-term return rates, or repeated exposure effects. As a result, policies optimized under these objectives may extract short-term revenue at the cost of gradual engagement degradation that accumulates over weeks or months and falls outside the observation window of typical A/B tests [37, 55]. Additionally, none of the formulations account for session-level dynamics: the reward at each step is evaluated independently, without modeling how a sequence of ad exposures within a session compounds to affect the final user state. LinkedIn's constrained formulation comes closest to capturing session-level budget effects through its engagement constraint C, but this is a per-decision threshold rather than a cumulative session-level model.

Reported results. Zhao et al. [101] report that their DEAR system, deployed on TikTok's short-video feed, achieves significant improvements in ad revenue without statistically significant degradation in organic engagement metrics in online A/B tests, though exact magnitudes are not publicly disclosed. Zhao et al. [103] similarly report that their jointly-trained RL policy outperforms baseline bandit approaches in both revenue and engagement metrics in live experiments, with the relative weights α and β validated across a sequence of experiments. Zhang et al. [99] report that their whole-page optimization at Alibaba yields improved revenue while maintaining Click Yield above the specified threshold T in production, demonstrating that a constrained formulation can be practically deployed at scale. Yan et al. [94] report improvements in both revenue and user engagement metrics at LinkedIn's feed, with the shadow bid approach enabling the system to adaptively balance monetization and organic quality as traffic conditions change.

Guidance for practitioners. The choice of utility formulation involves a fundamental trade-off between simplicity, revenue fidelity, and organic representativeness. The binary reward formulations (DEAR, Zhao et al. 2020) are the easiest to implement and require no revenue model, making them suitable for early-stage systems or platforms where revenue attribution is difficult. However, their coarseness limits optimization precision. The monetary utility formulations (Alibaba, LinkedIn) more faithfully represent actual platform revenue objectives, but they depend on accurate pCTR models and require setting additional hyperparameters such as the Click Yield threshold T or the regularization weight w. The constrained formulation of Alibaba is particularly well-suited when engagement is treated as a non-negotiable floor, since it directly encodes that requirement without requiring a precise conversion factor between engagement and revenue. LinkedIn's shadow-bid approach is appropriate when the conversion factor itself is a key design variable and is expected to vary across user segments or over time. Across all four approaches, none addresses long-term ad fatigue directly; practitioners operating in contexts where user retention is a primary concern should augment these formulations with explicit long-term holdout metrics [37] or consider incorporating delayed engagement signals into the reward function [55].

4.3 Action Space Representation

This subsection examines how action-space design determines which ad-policy levers an RL policy is allowed to control. We synthesize four structural properties that we identified across industrial and academic ad-policy systems: expressiveness, orthogonality, interpretability, and outcome predictability. These properties are an authorial taxonomy distilled from recurring design considerations rather than a field-wide taxonomy inherited from a single source. We first translate the action space into ad-policy controls, then analyze the trade-offs implied by each property, and finally summarize which production levers practitioners should prioritize and why.

In RL, the action space $\mathcal{A}$ denotes the set of choices available to a policy at each decision point [77]. In monetization-aware ranking, those choices specify how ads are inserted into a recommendation slate: whether to show an ad, which ad to select, and where it should appear relative to organic content [40, 94]. Selecting among these actions aims to maximize platform revenue while maintaining or improving user engagement [103]. Although $\mathcal{A}$ is technically discrete, the number of combinations grows exponentially with the number of candidate ads and available slots. Enumerating or evaluating every action individually is therefore computationally infeasible, and the space must be treated using methods designed for high-dimensional discrete decision problems [26, 39]. Narrowing $\mathcal{A}$ to a tractable subset involves trade-offs that the four properties below make explicit.

Industrial systems usually reduce this action space before policy learning rather than optimizing directly over every feasible slate. Candidate-generation and ranking stages first restrict the set of ads or organic items available for a decision [20, 21, 104]. These reductions improve tractability, make it more likely that historical logs contain enough

examples to estimate action outcomes, and reduce serving latency, but they also remove some globally feasible actions from consideration, so the learned policy can only be optimal within the reduced action family.

Expressiveness. An action space should expose the platform interventions that materially affect the revenue–engagement trade-off, such as ad load, placement, eligibility, pricing thresholds, or slate composition. Greater expressiveness gives the policy more control, but it also increases the number of state-action configurations that must be estimated from logged data.

Orthogonality. Action dimensions should be separable enough that operators can attribute policy changes to specific levers. In ad systems, however, levers such as load, placement, and ad quality are often coupled, so perfect orthogonality is rarely achievable.

Interpretability. Actions should have enough semantic meaning that engineers, product owners, and policy reviewers can reason about what the learned policy is doing before full deployment. Highly abstract slate representations can enlarge the policy class, but they also make it harder to explain why a particular ad was inserted, suppressed, or moved.

Outcome Predictability. Taking the same action in the same state should yield statistically predictable outcomes over many episodes [63]. The expected engagement and monetization outcomes for a given state-action pair should be stable enough that repeated exposure enables accurate estimation of long-term value [56, 100]. In practice this assumption is violated by non-stationary user preferences, seasonality, and auction dynamics [26, 37]. Systems address the gap with different mechanisms: off-policy evaluation with importance weighting [78], off-policy bandit learning that re-estimates under the current logging policy [68], and frequent retraining on rolling windows in deployed platforms [34]. The trade-off is between the data efficiency gained from assuming stationarity and the drift risk accumulated over long training windows.

Table 7 summarizes the concrete action spaces used by four representative systems before returning to the four-property comparison.

The comparison shows the central action-space trade-off: richer actions give the policy more control over ad insertion, but they also increase the number of state-action outcomes that must be estimated from logged data. Slate-level and page-level actions can represent interactions among items and positions, while lower-dimensional placement or allocation controls are easier to evaluate but restrict what the learned policy can change.

In production, these properties are operationalized through concrete ad-policy levers such as ad load, ad quality, ad placement, and ad relevance [46]:

- **Ad Load.** The frequency or spacing of ads in the feed. Placing ads too closely together can lead to user fatigue and reduced click probabilities [1, 103].
- **Ad Quality.** Intrinsic features of the ad, such as design format, product type, accompanying text, and landing-page quality, all of which influence user engagement [60, 93].
- **Ad Placement.** The position of the ad within the feed. Ads placed earlier are more likely to be clicked, though relevance to surrounding content also plays a significant role [69, 97, 103].
- **Ad Relevance.** The alignment between the ad and user preferences or intent. Ads shown adjacent to related organic content, such as a hotel ad next to vacation-related search results, are more likely to drive engagement [16, 106].

Figure 2 illustrates a concrete example of such an action set, showing how these ad-policy levers combine into the set of choices available to the policy at a single ranking opportunity.

Paper / system	Action-space definition	Commentary across the four properties
DEAR (ByteDance / Douyin) [101]	$a_t = (a_t^{ad}, a_t^{loc}) \in \mathcal{A}$, where t indexes the request, $\mathcal{A}$ is the action space, a_t^{ad} is the candidate ad, and $a_t^{loc} \in \{0, \ldots, L+1\}$ is the insertion location in a list of length L; location 0 denotes no ad. The action therefore decides whether to insert one ad, which ad to insert, and where to place it.	Strong interpretability and outcome predictability because the action has a direct ad-placement meaning and can be evaluated from logged ad-location outcomes. Expressiveness is limited by the at-most-one-ad formulation, and orthogonality is partial because ad choice and insertion location interact in the Q-value.
SlateQ (Google / YouTube) [40]	$A \in \mathcal{A}$ is a length-k slate drawn from item set I; in the unordered case, $A \subseteq I$ and $\lvert A\rvert = k$. Here, A is the recommended slate, $\mathcal{A}$ is the set of feasible slates, I is the item universe, and k is the slate size.	Highly expressive for slate-level recommendation because it preserves item co-presentation. Tractability comes from decomposing $Q^\pi(s, A)$ into item-level values, where Q^π is the value of slate A in state s under policy π, but outcome predictability depends on user-choice assumptions.
LinkedIn feed-ad allocation [94]	x_i indicates whether feed position i receives an ad; the threshold rule sets $x_i = 1$ when $r_i^a + \alpha u_i^a - \alpha u_i^o > 0$. Here, r_i^a is ad revenue utility, u_i^a is ad engagement utility, u_i^o is organic engagement utility, and α is the Lagrange multiplier trading engagement against revenue.	Interpretable because α acts as an engagement-revenue exchange rate, and outcome predictability is supported by a constrained allocation rule. Expressiveness is moderate: the policy controls ad insertion positions after separate rankings, but it does not jointly rerank all ad and organic candidates.
Alibaba whole-page optimization [99]	$a = (x_1, \ldots, x_K)$, where a is the page-level action, K is the number of page slots, and $x_k = 1$ displays an ad in slot k while $x_k = 0$ leaves that slot non-ad. The action determines which slots on the current page are filled by ads.	More expressive than slot-independent insertion because the action is page-level and can capture cross-position effects. The cost is lower orthogonality and interpretability: performance is attributed to a coupled page allocation rather than a single independent placement lever.

Table 7. Concrete action-space definitions and property-based critique across representative ad-policy and slate-recommendation systems. The table reports each paper's action definition or corresponding operational decision, interprets what the action means, and summarizes how the design performs against expressiveness, orthogonality, interpretability, and outcome predictability.

Taken together, the four properties and their associated trade-offs provide a structured way to compare published systems and to reason about which design choices are appropriate for a given deployment context. The next subsection turns to the state space, which defines the context in which these actions are taken.

4.4 State Space Representation

The preceding subsection examined how to structure the action space. This subsection addresses the corresponding question for the state space in policies that jointly rank ads and organic content in large-scale recommendation systems: what information must the state encode so that the policy can balance short-term monetization against long-term user engagement across successive interactions? To compare representative systems, we use five structural properties as an analytical lens: Markovianity, expressiveness, stability under distribution shift, sample efficiency, and interpretability. As with the action-space taxonomy in the preceding subsection, this set is synthesized from recurring design considerations across the systems reviewed here rather than inherited from any single source or presented as a field-wide consensus taxonomy. The subsection first defines the state space formally, then analyzes each property with the trade-offs and design choices it implies, and finally summarizes how representative systems operationalize them.

In RL, a *state* $s \in \mathcal{S}$ denotes the information on which the agent conditions when selecting an action. Together with the action space $\mathcal{A}$, a transition kernel $P(s' \mid s, a)$, a reward function $R(s, a)$, and a discount factor $\gamma \in [0, 1)$, the state space is one component of the Markov decision process (MDP) used as the standard formal framework for RL [63, 77]. In monetization-aware ranking, a single state instance typically aggregates user-level signals (e.g., identifiers, demographics, long-term preference embeddings), session-level context (e.g., recent engagement, dwell time, device), and platform-level features (e.g., current ad load, inventory composition, the slate rendered so far) at the time of a

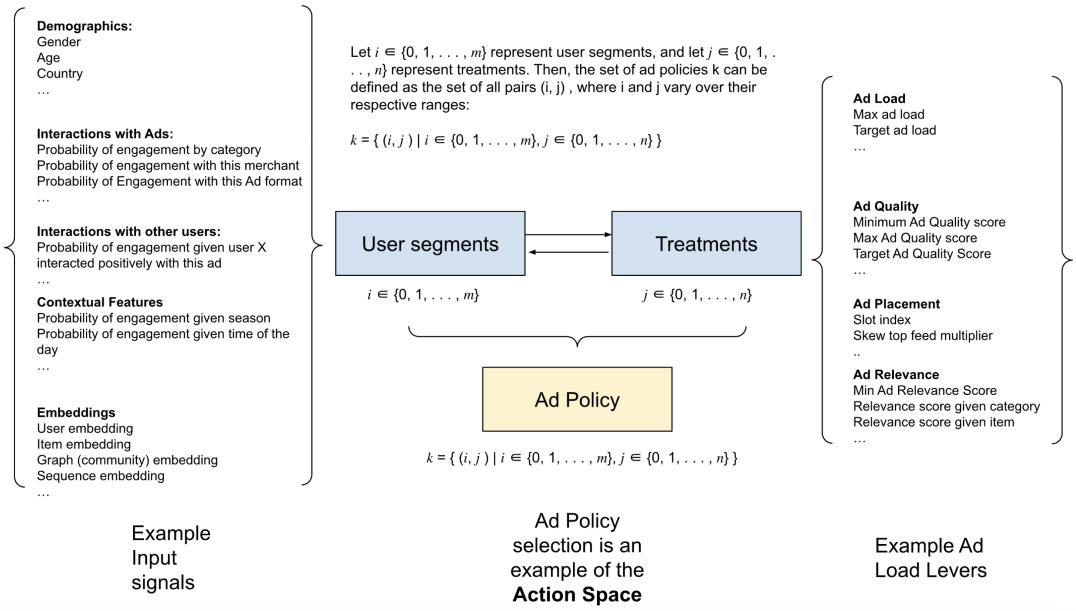

Fig. 2. Example of an action set in the context of ad policies in recommendation systems

decision [40, 91, 101]. Because only a subset of the latent user intent and platform dynamics is observable through these signals, constructing s is itself a design problem.

Markovianity. A state is Markovian when it contains everything needed to predict the next user-level outcome and monetization signal given the chosen action, with no additional information to be gained from the prior trajectory [63]. Markovianity matters because the value estimators, off-policy methods, and policy-gradient updates used to learn ad ranking decisions all assume the next reward depends only on (s, a). When it does not, $Q(s, a)$ averages over unobserved trajectory states, increasing the variance of counterfactual value estimates learned from logged feedback [78]. In ad policies, exact Markovianity is generally unrealistic: user intent and long-term satisfaction are partially observable, seasonality and auction dynamics shift over days to weeks, and new inventory arrives continuously [26, 37]. The practical question is therefore how Markovian the state must be for those measurements to remain stable across retraining cycles. Published systems address this issue by extending the state with longer behavioral histories (Deep Interest Network (DIN) [106], PinnerFormer [62]) or by compressing long histories into fixed-size representations through self-attention modules (Self-Attentive Sequential Recommendation (SASRec) [44], TransAct [91]). The trade-off is between the bias introduced by ignoring trajectory history and the sample complexity added by enlarging the effective state dimension.

Expressiveness. A useful state representation should encode the axes a joint ad / organic policy needs in order to distinguish between users, sessions, and slate contexts. In monetization-aware ranking, expressiveness has three components: user-level signals that separate long-term commercial intent from casual browsing (PinnerFormer [62], DIN [106]), session-level signals that capture short-term context such as recent dwell or query intent (TransAct [91]), and platform-level signals that describe what has already been rendered on the page, including the slate-so-far state used by SlateQ [40] and the request-level features used by DEAR [101]. Adding axes can broaden the policy class,

but it also enlarges s, which raises serving latency, model size, and the variance of the off-policy gradient estimator [52, 59, 78]. The trade-off is between covering the decision-relevant variation in the population and keeping s small enough that value estimates remain statistically stable.

Stability under distribution shift. The state representation should remain informative as the logging policy, ad inventory, and user behavior change between training cycles [32, 37]. One recurring pattern in large-scale recommender systems combines slower-changing user or item representations with features refreshed closer to serving time, such as recent engagement, trending content, or current ad load [52, 91, 98]. The slower-changing component is intended to smooth short-horizon noise and remain stable across retraining cycles, while the near-real-time component captures drift fast enough that the policy does not become stale between deployments. Representations that lean entirely on dense online features chase short-term signal at the cost of unstable Q-value estimates across A/B windows; representations that lean entirely on offline features are robust to drift but slow to react to new inventory or campaign launches. The trade-off is between responsiveness to behavioral drift and the stability of value estimates over the retraining horizon.

Sample efficiency. Off-policy estimators, policy-gradient updates, and counterfactual evaluators all require enough (s, a, r) tuples per region of state space to estimate $Q(s, a)$ with controllable variance [49, 78]. As the dimensionality of s grows, the effective sample density per region drops, inflating the variance of importance-sampled estimates and slowing actor convergence [26]. Engineered low-dimensional features such as commercial-intent scores, ad-load history, or dwell quantiles can converge faster from the same logged volume, but they encode the designer's prior about which axes matter, and that prior bounds the policy class [101]. The trade-off is between the bias accepted from feature engineering and the sample volume required to learn over a higher-dimensional learned representation.

Interpretability. State components with human-readable meaning let operators trace policy behavior to upstream signals, support pre-launch review, and isolate regressions when A/B lifts move [34, 35]. Engineered features such as commercial-intent scores, recency buckets, or ad-load counters allow a reviewer to predict roughly how the policy will respond before traffic is sent. Embedding-dominated states such as those used by PinnerFormer [62] and TransAct [91] often expose no low-dimensional summary an operator can read, so behavioral validation defers to live A/B measurement. The trade-off is between the breadth of behavior the state can support and the ability of operators to vet the policy before deployment, mirroring the same trade-off identified in the action-space subsection.

Table 8 summarizes how four representative systems operationalize the five properties. As with the action-space comparison in the preceding subsection, no system satisfies all five cleanly; each accepts an explicit trade-off based on its deployment context.

Reported results. Zhao et al. [101] evaluate DEAR on TikTok's short-video feed and report that the request-level state representation supports ad revenue improvements without statistically significant degradation in organic engagement in online A/B tests, though exact magnitudes are not publicly disclosed. Ie et al. [40] evaluate SlateQ in the RecSim simulator [39] before deploying to a YouTube-scale service, and report that state representations including the slate-so-far context yield gains on a long-term user satisfaction metric relative to myopic slot-level baselines. Pancha et al. [62] evaluate PinnerFormer at Pinterest against prior user-embedding baselines and report that a single user embedding trained on a multi-week engagement sequence yields meaningful offline retrieval-quality gains and online engagement lift across Home feed and related surfaces. Xia et al. [91] evaluate TransAct at Pinterest as a ranking-stage augmentation over PinnerFormer, and report that combining the long-horizon embedding with a real-time session sequence yields additional engagement lift, with the offline component absorbing long-term user signal and the real-time sequence capturing immediate intent. Zhou et al. [106] evaluate DIN on Alibaba display advertising and report area under the ROC curve (AUC) improvements over a deep CTR baseline by attending to prior user behavior conditional

Property	DEAR [101]	SlateQ [40]	PinnerFormer [62]	TransAct [91]
Markovianity	Request-level features with short engagement history	User context with slate-so-far; user-choice model assumed	Fixed-size embedding summarizing multi-week engagement trajectory	Long-term embedding combined with real-time session sequence
Expressiveness	Engineered low-dimensional per-request features	Full slate with per-item decomposition	Single user embedding capturing long-term preference structure	Session sequence plus long-term embedding covering both horizons
Stability under distribution shift	Drift absorbed through frequent retraining on rolling windows	Simulator-based training; drift handling not explicitly reported	Daily batch refresh; stable across retrain cycles, slow to inventory shifts	Explicit offline / online split, directly targeting drift
Sample efficiency	Low-dimensional state converges quickly from logged data	Per-item Q decomposition reduces sample complexity per slate	Pre-trained embedding shares signal across downstream tasks	Self-attention compression controls cost of higher state dimensionality
Interpretability	Engineered features readable by operators	Per-item Q-values partially readable; slate behavior not	Embedding-only; opaque to pre-launch review	Embedding-dominated; behavioral validation requires A/B
Primary trade-off	Simplicity vs. coverage of long-term behavior	Choice-model assumption vs. slate-level expressiveness	Long-horizon stability vs. responsiveness to fresh inventory	Expressiveness and stability vs. serving latency and model size

Table 8. Comparison of state-representation design choices across representative ad-recsys systems. Each row corresponds to one of the five structural properties introduced in this subsection; each column describes how a published system operationalizes that property. The final row states the primary trade-off each system accepts as a consequence of its choices.

on the candidate ad. Across these reports, state-representation choices are evaluated through a combination of offline replay on logged interactions, simulator-based ablation, and live A/B measurement; in our review of these papers, none share random seeds, hyperparameter search spaces, or full reproducibility artifacts, which limits cross-system comparison to the qualitative axes in Table 8.

Before discussing the levers that shape these properties, we summarize the representational building blocks that s is typically assembled from in ad + organic systems. *User embeddings* encode long-term commercial intent, preference structure, and behavioral regularities (PinnerFormer [62], DIN [106]); they give the policy a population-level prior over which ad categories are relevant before any session-specific signal arrives. *Ad features* encode sparse attributes such as ad identity, campaign, product category, or advertiser context, helping the policy generalize across ads with limited interaction history [20, 56]. *User-ad interaction features*, whether computed as dot products, cross features, or attention over recent impressions [20, 106], expose the fit between a specific ad and the current user; they are the axis most directly predictive of click and conversion and are the one most sensitive to distribution shift. *Contextual signals* (location, device, time, slate-so-far) anchor the representation when longer histories are weak or absent [101]. Each lever below acts on one or more of these building blocks.

The five properties are operationalized in production through a small set of state-representation levers that recur across published systems:

- **Long-history user embeddings.** Extend s along the temporal axis to reduce history dependence and broaden expressiveness, at the cost of sample efficiency (PinnerFormer [62], DIN [106]).
- **Sequence compression.** Apply self-attention modules to reduce a long behavioral history to a fixed-size summary, recovering sample efficiency without discarding the temporal signal (SASRec [44], TransAct [91]).
- **Offline / online representation decomposition.** Combine slow-changing batch-computed representations with near-real-time user or context features to target stability under distribution shift while controlling serving latency [52, 91, 98].

- **Engineered contextual features.** Anchor s with low-dimensional human-readable signals (location, device, ad-load history, onboarding interests) to support interpretability and address the cold-start regime when long histories are unavailable [21].

These levers are typically combined: heterogeneous embeddings are concatenated and projected through learned linear layers into a shared dimension [21, 36, 62], with zero-padding and truncation avoided because they inject noise or cause information loss [91]. The set of levers a system enables, rather than any single architectural choice, determines where it lands on the property trade-offs above.

Guidance for practitioners. The choice of state representation involves a trade-off between expressiveness, sample efficiency, serving cost, and the ability to audit policy behavior before deployment. Engineered low-dimensional states such as those used in DEAR [101] are appropriate for early-stage systems, surfaces with limited logged volume, or deployments where pre-launch interpretability is a regulatory or product constraint; their sample efficiency and readability come at the cost of coverage of long-term user behavior. Slate-so-far representations with user-choice decomposition, as in SlateQ [40], are appropriate when the action space is genuinely slate-level and the user-choice assumption is defensible for the target surface; they extend expressiveness without forcing exhaustive enumeration of slates, but require simulator or counterfactual infrastructure to validate the user-choice model itself. Long-horizon user embeddings such as PinnerFormer [62] and DIN [106] are appropriate when long-term engagement is the primary objective, the platform has sufficient daily active users to stabilize embedding training, and mature batch-compute infrastructure is already in place; practitioners operating below this scale should expect the sample-efficiency trade-off to dominate. Hybrid offline / online representations such as TransAct [91] and large-scale online user-representation systems for ads personalization [98] are appropriate for mature production systems where both drift responsiveness and long-term signal matter, and where the serving stack can accept the latency overhead of real-time sequence encoding. Among the systems compared here, none directly addresses the joint ad + organic-content attribution problem in its state design; practitioners optimizing for the balance between monetization and organic engagement should augment the state with organic-slot context (surrounding organic quality, recent organic dwell) or combine state-space extensions with the constrained-reward formulations discussed in Section 4.3 [94, 99].

Taken together, the five properties, the comparison in Table 8, and the levers above provide a structured way to compare how published systems construct s and to reason about which representational choices are appropriate for a given deployment context. The next subsection turns to policy learning, which determines how the agent uses (s, a, r) tuples drawn from this state space to update its decision rule.

4.5 Policy Learning

This subsection defines the ad-serving policy, contrasts deterministic and stochastic formulations, and organizes policy learning around two recurring distinctions: offline versus online learning, and value-based versus policy-based optimization. For each distinction, it discusses advantages, limitations, and practitioner considerations for when one approach may be preferred over another, with examples of hybrid approaches used in large-scale recommendation and ad-serving systems.

In the context of ad policy optimization, the policy $\pi(a|s)$ maps a state $s \in \mathcal{S}$ to an action $a \in \mathcal{A}$ in order to maximize expected reward [77]. The action a represents the decision available to the policy, which may encode a concrete ad or slate selection, or a control lever such as ad load, placement, format, or pricing threshold, as detailed in Section 4.3 [40, 99, 101]. The state s represents the information available to the policy when making that decision, which may

include the user's profile, browsing history, user-item interactions, session context, and inferred commercial intent, as discussed in Section 4.4 [62, 91, 105]. Policies differ in how they choose among candidate actions: a deterministic policy maps the same state to the same action, whereas a stochastic policy samples an action from a probability distribution over candidate actions [77].

In many large-scale recommendation and ad-serving systems, learned policies are stochastic or include stochastic exploration mechanisms, whereas simpler or older systems may use deterministic rules, such as inserting a fixed ad after a fixed number of organic impressions [34, 101]. Stochastic policies are useful because they support exploration: the system can sample from multiple plausible actions rather than always selecting the historically best-performing one [66, 77]. This can help the system continue exploring uncertain user-ad or user-slate matches, rather than locking prematurely onto the current best estimate [10, 66, 101]. Such exploration is especially relevant when feedback is sparse, delayed, or tied to long-term outcomes [54, 109]. Stochastic policies also provide a practical mechanism for decision-making under uncertainty and partial information [26, 66]. When objectives such as monetization and long-term engagement conflict, randomized or probabilistic action selection can preserve learning opportunities while the system estimates which actions perform best under different contexts [10, 41, 57]. Deterministic policies can encode complex tradeoffs, but they may require manual retuning as user behavior, inventory, or auction conditions shift. Stochastic policies, by contrast, can update action probabilities as additional feedback becomes available [34, 66].

4.5.1 Offline vs Online Policy Learning. This subsubsection explains why large-scale ad systems often begin with offline policy learning before introducing online updates, and compares the safety, scalability, and adaptation trade-offs of the two approaches. Depending on whether the agent learns by training on historical data or by interacting with the environment in real time, policy learning can be categorized as either offline or online [77]. In offline policy learning, models are trained on logged interactions before they are exposed to live traffic. This makes offline learning a common starting point for large platforms because it supports policy screening, counterfactual evaluation, and long-horizon reward estimation before exposing a candidate policy to live traffic [19, 34, 49].

For example, an ad-ranking policy can be trained from historical impression logs that record the user state, the ad or slate shown, and later reward signals such as clicks, conversions, revenue, or retention proxies [19, 78, 101]. Methods such as Monte Carlo return estimation are useful when the relevant outcome is delayed, but they depend on complete and representative logged trajectories and can become sample-inefficient or high-variance when feedback is sparse [26, 49, 77]. Thus, offline learning is most useful as a risk-controlled evaluation and initialization stage, not as a substitute for carefully staged online validation.

Logged data only contains actions chosen by the historical policy, which means there is no direct visibility into what would have happened if alternative actions had been taken in the same context [78]. This lack of counterfactual information makes it difficult to evaluate or iterate on new policies. However, methods such as inverse propensity scoring (IPS), which re-weights logged rewards to approximate outcomes under a different policy, and doubly robust estimators, which combine IPS with model-based value predictions, can help address this challenge [25, 49, 78]. These approaches enable limited forms of counterfactual reasoning, especially when the new policy does not diverge too far from the one that generated the data.

In ad policy optimization, this limitation arises when the log contains little or no data for a candidate ad action in a given user context. A function approximator trained over related state-action pairs may still estimate the action's expected return. It does so by generalizing from similar users, sessions, ads, and placements. These estimates can

support limited counterfactual reasoning when the target action is close to regions with sufficient logged support. If the logged data contains few similar examples, however, the estimate becomes much less reliable [49, 78].

Online policy learning complements offline learning by collecting new feedback for candidate actions that were rarely or never selected by the previous policy. In practice, large platforms rarely expose an unconstrained RL agent directly to all traffic. They usually introduce online learning through controlled mechanisms such as contextual-bandit updates [10, 50, 57, 81], staged traffic ramps, feature gates, or actor-critic policies monitored with guardrail metrics [19, 34, 92]. These mechanisms allow the system to test candidate ad-load, placement, or ranking actions, observe fresh click, conversion, revenue, and engagement feedback, and update the policy as user behavior, inventory, or auction conditions change. The benefit is adaptivity; the cost is that exploration can temporarily serve suboptimal ads or ad loads, so online learning is typically used after offline evaluation and under rollout controls [19, 26, 34, 92].

To mitigate risks and costs of online exploration, large-scale systems commonly begin with offline training or offline evaluation before controlled online rollout [19, 34, 49]. Online policy updates are introduced gradually via controlled bandit loops, feature gates, or soft rollouts. This hybrid approach of offline-then-online policy learning is commonly seen in industry. In their offline RL system, YouTube trained an actor-critic model entirely on logged user interaction data, using Monte Carlo returns to estimate long-term value, and only deployed policies to production after thorough offline evaluation [19]. At Meta, the Horizon platform first trains policies offline and uses counterfactual policy-evaluation methods, including doubly robust estimators, before deploying policies through gated rollouts, which refer to the controlled and staged release of new policies to a limited portion of traffic in order to minimize risk during online learning [34]. Spotify similarly adopts a cautious approach by first optimizing contextual bandit policies offline, and then introducing online learning through incremental updates and careful exploration strategies [57]. Table 9 summarizes the practical distinction between offline learning, online learning, and the hybrid rollout pattern that commonly connects them.

Pattern	Role in ad-policy learning	Typical mechanism	Main trade-off
Offline learning	Screen and initialize candidate policies before live exposure [19, 34, 49]	Train on logged impressions, estimate long-horizon returns, and use counterfactual policy evaluation [19, 25, 78]	Lower deployment risk, but estimates are unreliable for actions rarely selected by the previous policy [49, 78]
Online learning	Collect fresh feedback for candidate ad-load, placement, or ranking actions [10, 50, 81]	Use contextual-bandit updates, guarded policy updates, or on-policy RL under monitoring [34, 57, 92]	More adaptive to changing users, inventory, and auctions, but exploration can temporarily serve suboptimal ads or ad loads [26]
Offline-then-online rollout	Combine offline screening with limited live validation before broad deployment [19, 34, 57]	Start with offline training or evaluation, then introduce updates through feature gates, staged traffic ramps, or soft rollouts [19, 34]	Balances safety and adaptivity, but requires monitoring, guardrails, and careful rollout design [26, 34]

Table 9. Practical comparison of offline, online, and hybrid policy-learning patterns in ad-policy optimization.

4.5.2 Value-based vs Policy-based Learning. This subsubsection compares value-based, policy-based, and hybrid policy-learning methods, highlighting their trade-offs and when each is most useful for ad-policy optimization in industrial recommender systems.

RL algorithms can be divided into value-based and policy-based methods based on how they learn to make decisions to maximize cumulative rewards [77]. Value-based methods first estimate the value of states or state-action pairs and then derive a policy from these estimates, whereas policy-gradient methods learn a parameterized policy by optimizing

expected cumulative reward directly [87]. In other words, value-based methods focus on learning the utility of states or actions and derive the optimal behavior indirectly by acting greedily with respect to this learned value. Policy-gradient methods, in contrast, optimize the policy itself, although hybrid actor-critic methods often add a value estimate to stabilize learning [19, 58].

Because value-based methods rely on greedy action selection to determine the next best action, they require comparing the estimated value of possible actions at each decision point [77, 108]. As the number of available actions grows, and as the agent encounters more states during interaction, this comparison becomes harder to scale [26]. For this reason, value-based methods are particularly well-suited for environments with discrete or low-dimensional action spaces, where scoring candidate actions and comparing their estimated values remains computationally feasible [108]. In ad-policy settings, value-based formulations appear in personalized ad recommendation, sponsored-search real-time bidding, and budget-constrained display advertising, where the learner estimates the long-term value of candidate ad actions before deriving a policy [80, 88, 100].

In policy-based learning, the agent directly adjusts the decision rule used at each decision point to maximize expected cumulative reward, rather than first estimating the value of every candidate action at that decision point and then choosing the action with the highest estimated value. Formally, the goal is to learn a parameterized policy $\pi_\theta(a \mid s)$, where θ denotes the parameters of the policy [87]. In large-scale recommendation and ad-serving systems, these parameters are often the weights of a model that maps user, item, and context features to action probabilities [19, 101]. Neural network-based policies are common in modern industrial RL and recommendation settings due to their expressiveness and scalability, making them particularly well-suited for environments with complex and high-dimensional state spaces [51]. Formally, the objective function to be maximized is the expected cumulative reward under the policy:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] \tag{1}$$

where τ denotes the user or session trajectory induced by the policy, including the sequence of observed states, selected ad-policy actions, and resulting feedback signals [19, 101]. Policy-gradient methods estimate the gradient of the expected return with respect to the policy parameters [77, 87], commonly written as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s) \cdot Q^{\pi_\theta}(s, a)\right] \tag{2}$$

where $Q^{\pi_\theta}(s, a)$ is the action-value function under the current policy [77]. In ad-ranking systems, this value term can be interpreted as an estimate of the downstream return associated with selecting a candidate ad-policy action in the current user or session context [19, 101]. Since the goal is to maximize return, gradient ascent, not descent, is used [87]. Common policy-based RL algorithms include REINFORCE, Proximal Policy Optimization (PPO), Trust Region Policy Optimization (TRPO), and actor-critic methods, which combine policy and value learning in a hybrid architecture [58, 70, 71, 87].

Policy-based methods can be more useful when exploration should vary by user or context, because the policy directly learns the parameters of an action distribution rather than relying on a fixed exploration rule. Unlike a greedy value-based policy that selects $\arg\max_a Q(s, a)$, a parameterized ad policy can assign probabilities directly to candidate actions such as ad-load levels, insertion positions, bids, or slate choices in the current user and auction context [92, 101]. Value-based systems can still explore through mechanisms such as ϵ-greedy action selection, but the exploration rule is usually specified separately from the learned value estimates [26, 43].

Policy-based methods also have important limitations. Gradient estimates from sampled trajectories can have high variance, which can lead to unstable learning and slow convergence [70, 87]. Because pure policy-gradient methods do not learn an explicit value function for action evaluation, they can also require more samples than value-based methods in large-scale environments [26]. To address these limitations, many large-scale production systems adopt hybrid approaches that combine direct policy learning with value estimation, most notably actor-critic architectures [19, 34]. In these methods, the actor learns the policy while the critic provides lower-variance value estimates that stabilize policy updates, managing the bias–variance trade-off without giving up direct optimization of the policy objective [19, 58].

Put simply, value-based and policy-based methods differ in what the model learns. Consider a request in which the system must choose between two ad-load actions: in a simplified hypothetical setting, a low ad load might mean ads occupying 10% of the feed slots, whereas a high ad load might mean ads occupying 50% of the feed slots. A value-based method first estimates the expected return of each action in the current user context s, for example $Q(s, \text{low ad load})$ and $Q(s, \text{high ad load})$, and then derives a policy by choosing the action with the larger estimated value. A policy-based method instead trains a model whose direct output is the action distribution $\pi_\theta(a \mid s)$; for the same request, the model might assign 0.75 probability to low ad load and 0.25 probability to high ad load.

The value-based vs policy-based taxonomy offers a useful lens for categorizing RL approaches in ad policy optimization, based on how decisions are learned to maximize cumulative rewards [51, 101]. Large-scale ad systems such as Google Ads have long relied on supervised prediction models for immediate outcomes such as click prediction [56]. Value-based RL formulations, by contrast, have been explored for personalized ad recommendation, sponsored-search real-time bidding, and budget-constrained display advertising, where the learner estimates the long-term value of candidate ad actions before deriving a policy [80, 88, 100]. It is also possible to optimize for the ad policy directly in a more flexible formulation [87]. For instance, rather than predefining a limited set of treatment arms, TikTok used Proximal Policy Optimization (PPO) to learn a stochastic ranking policy that directly maps user context to slates of ads [101]. This approach can handle potentially large or complex sets of ads more flexibly, allowing for direct parameterization of the stochastic policy [26, 40].

There are well-documented industry examples of the use of actor-critic methods in ad recommendations. For example, Facebook reported that RL models (including off-policy actor-critic methods) trained on its Horizon platform outperformed and even replaced previous supervised recommendation systems in tasks like notifications and video ranking [34]. YouTube had previously used REINFORCE, a policy-gradient method that updates the policy from full-trajectory returns and therefore tends to produce high-variance gradient estimates when training from logged data [19, 87]. They replaced it with an off-policy actor-critic approach that adds a critic network to estimate Q-values via temporal difference (TD) learning [19]. The use of Q-value estimates enables lower-variance policy updates compared to Monte Carlo returns; one-step importance correction offers more robust handling of off-policy data than full-trajectory importance sampling; and bootstrapping with one-step TD targets improves the estimation of long-term rewards [19].

Actor-critic methods are particularly useful in multi-objective settings where both revenue and user engagement must be balanced, and where using logged data for offline training is desired [13]. To this end, two-staged constrained actor-critic methods have been effectively used to optimize the main objective while enforcing guardrail constraints through a constrained MDP formulation. In stage one, multiple actor-critic agents learn to optimize auxiliary objectives (e.g. engagement such as likes, saves, and shares). In stage two, a primary actor-critic learns a policy maximizing the main objective (e.g. revenue, ad clicks) while staying close to the auxiliary policies to satisfy those constraints. This

two-tier actor-critic approach achieved a superior balance: offline experiments showed it outperformed alternative RL methods on the trade-off between watch time and other interaction metrics [13].

Across these approaches, the structure and cardinality of the action and state spaces, the calibration of exploration vs exploitation, the available computational budget, and the degree of cohesion required across an ML pipeline are some key factors that help determine which paradigm to use in practice [26, 108]. Table 10 summarizes these practical considerations across the main policy-learning approaches discussed in this subsection.

Approach	When preferred	Main advantage	Main limitation
Offline learning	Before live rollout, when exploration is risky or costly [19, 34, 49]	Safer policy screening using logged data	Sensitive to logging-policy bias and limited support [25, 78]
Online learning	After offline validation, when fresh feedback is needed [34, 57]	Adapts to changing users, inventory, and auctions	Can expose users to suboptimal policies during exploration [26]
Value-based learning	Small or enumerable action spaces [80, 108]	Sample-efficient action-value estimation	Harder to scale to large slates or integrated neural ranking policies [26, 40]
Policy-based learning	Stochastic policies, large action spaces, or learned slate-ranking policies [92, 101]	Directly parameterizes the decision rule	Often less sample-efficient and higher variance [26, 70]
Actor-critic hybrids	Large-scale systems needing both direct policy learning and stable value estimates [19, 34]	Combines policy flexibility with value-based stabilization	More complex to train, tune, and evaluate [26]

Table 10. Practical comparison of policy-learning approaches for ad-policy optimization.

4.6 Balancing Exploration and Exploitation in Policy Learning

Balancing exploration and exploitation is a fundamental challenge in policy learning [77], particularly at large scale, whereby exhaustively exploring all options is computationally infeasible, the cost of suboptimal decisions compounds quickly, and sustained bias toward early winners can prevent the discovery of truly optimal policies over time [26].

In ad policy optimization, tuning exploration and exploitation is critical across multiple dimensions, notably ad selection and ad load. In *ad selection*, a system must decide between exploring new ads that might yield higher rewards and exploiting ads known to perform well [8, 10, 66]. In setting *ad load* configuration, platforms must weigh the exploration of higher ad exposure for queries with high commercial intent against the risk of degrading user experience [14, 37, 99]. While exploring more aggressive ad load configurations may unlock revenue gains, it risks triggering ad fatigue, reduced engagement, and eventual user attrition [67, 74].

In this section, we introduce the commonly used methods in industry as well as their pros and cons, and the contexts in which they are most effective.

4.6.1 ϵ***-Greedy Strategy.*** One of the simplest and most widely used exploration methods, the epsilon-greedy strategy allows the agent to primarily exploit known successful actions (with probability $1 - \epsilon$) while occasionally exploring random actions (with probability ϵ) [77]. Over time, ϵ *can* be reduced, shifting focus toward exploitation as the agent gains more experience [66]. However, deciding how much and when to decrease ϵ is non-trivial, as reducing exploration too quickly can lead to premature convergence on suboptimal actions [26].

ϵ-Greedy based exploration/exploitation is a relatively easy method to implement, computationally efficient, and thus well-suited for real-time applications [77]. However, because it treats all actions uniformly during exploration, ϵ-greedy does not differentiate between tried and untried actions, account for uncertainty in value estimates, or prioritize actions with higher potential expected improvement. This can lead to missed opportunities, especially in environments with

large action spaces, because as the number of possible actions increases, the probability of selecting any particular action during exploration decreases [11]. As a result, many potentially high-value actions may never be explored, or may be explored too infrequently to accurately estimate their value. ϵ-greedy is therefore most effective in smaller action spaces, where exploration remains tractable and the risk of encountering extremely low-value actions is lower due to the reduced likelihood of extreme outcomes [26].

4.6.2 ***Upper Confidence Bound (UCB).*** In contrast to ϵ-greedy strategies, which explore uniformly at random, confidence-bound methods form a family that ranks actions with a composite score: an exploitation term (estimated expected reward) plus an exploration bonus (uncertainty). Members of this family differ mainly in how they estimate the reward and its uncertainty: UCB1 uses the empirical mean and raw pull counts [8], linear UCB (LinUCB) fits a linear model over context features and derives the bonus from that model [50], and Regret Confidence Bounds (RegCB) trains a supervised model to predict each action's reward from its features and reads the uncertainty off that model's prediction error, so any regressor, including non-linear ones such as gradient-boosted trees, can drive exploration [31].

At each time step t, a UCB policy selects the action a that maximizes:

$$\mathrm{UCB}_t(a) = \hat{Q}_t(a) + \alpha \cdot \sqrt{\frac{\log t}{N_t(a)}} \tag{3}$$

where $\hat{Q}_t(a)$ is the estimated expected reward and the second term is an exploration bonus. In the finite-arm setting this reduces to UCB1: $\hat{Q}_t(a)$ is the empirical mean, $N_t(a)$ is the number of times a has been selected, and $\alpha > 0$ scales exploration [8, 11]. The bonus shrinks as $N_t(a)$ grows and the slow $\log t$ term sustains occasional exploration in later rounds, so an action is selected either because it looks rewarding (high $\hat{Q}_t(a)$) or because it is underexplored (high bonus); confidently poor actions are progressively deprioritized as their bonus decays [8].

This scoring pattern balances exploration and exploitation in the finite-arm setting. Because exploration is directed toward uncertain actions rather than sampled uniformly, UCB1 typically achieves lower long-run regret than fixed ϵ-greedy [8]. At any finite horizon, which method earns more reward depends on the chosen values of α and ϵ, not on the algorithm family alone [11].

In practice, confidence-bound exploration is most useful where the set of choices is small and candidate policies can be screened on logged data before launch. LinkedIn, for example, framed ad format and layout selection as a contextual bandit and compared UCB and Thompson Sampling offline through replay on historical ad-serving logs [79]. For budget pacing and cross-platform bidding, by contrast, the same platform relied on control- and optimization-based methods rather than bandit exploration: pacing throttled spend against a forecasted traffic curve [5], and bidding was cast as constrained optimization over advertiser value [33]. This contrast reflects a broader pattern in ad systems, where exploration bonuses help discover which ads perform well, while decisions about how much to spend and where to place ads are usually handled by constrained optimization once reward estimates are reliable.

UCB is most attractive when data is sparse and the optimization horizon is long, because its regret guarantees ensure that the cumulative cost of exploration grows slowly over time; this suits ad policy optimization aimed at long-term objectives such as user lifetime value or engagement [11, 80]. The same optimism that drives this behavior, however, means UCB deliberately samples actions whose value is uncertain, so it is less suitable where individual exploratory decisions are costly or where immediate results are prioritized; latency-sensitive or safety-critical settings favor conservative or risk-aware exploration instead [26, 56]. Furthermore, deploying UCB at scale introduces systemic

friction: it is more computationally expensive than ϵ-greedy approaches [10], and the choice of the exploration parameter α is brittle, as too conservative a setting limits the discovery of better actions while too aggressive a setting amplifies noise and degrades short-term performance.

4.6.3 ***Regret Confidence Bounds (RegCB).*** RegCB is another UCB-family variant that replaces LinUCB's linear reward estimate with a regression oracle, computing per-action confidence intervals from contextual features and supporting non-linear reward models [31]. Foster et al. show that this approach generalizes both UCB and LinUCB to richer model classes [31]. This makes it better suited than linear contextual UCB to the sparse, high-dimensional feature spaces typical of ad personalization, where the reward depends on non-linear interactions among user, ad, and context features [31]. In the Contextual Bandit Bake-Off, an offline benchmark across a large suite of datasets, regression-oracle methods in the RegCB family were among the most competitive exploration algorithms, though at higher computational cost than ϵ-greedy [10, 31]. Because this evidence comes from offline benchmarks rather than a reported production ad deployment, RegCB is best treated as a promising option for offline policy screening, with its online ad-policy performance not yet established in the sources reviewed here.

4.6.4 ***Thompson Sampling.*** Thompson Sampling (TS) uses a Bayesian approach in which actions are selected by sampling from the posterior distribution over rewards and choosing the action with the highest sampled value [66, 77]. Formally, the selected action at time t is:

$$a_t = \arg\max_{a \in \mathcal{A}} \tilde{Q}_t(a) \tag{4}$$

where $\mathcal{A}$ is the action set and $\tilde{Q}_t(a)$ is a sample from the posterior distribution of the expected reward for action a at time t [66].

While each individual decision is based on a single random sample, across many realizations, the likelihood that an action is selected reflects the posterior probability that it is the optimal choice [66]. Initially, TS relies on a prior distribution that reflects the initial belief about the reward of each action, usually based on some default assumptions or domain knowledge [66]. As more data is collected through interactions (e.g., clicks or conversions in an ad setting), the prior is updated to form a posterior distribution, representing the system's refined belief about expected rewards [66]. Actions with greater uncertainty (i.e., wider posterior distributions) are more likely to generate extreme reward samples, occasionally leading to their selection even if their mean reward is lower [66]. In contrast, actions with higher expected rewards and narrower posterior distributions are selected more consistently due to their reliably higher sampled values [66]. Thus, TS leverages posterior variance to simultaneously enable exploration and exploitation without requiring an explicit exploration bonus as used in UCB methods [66].

While both UCB and TS leverage estimates of variability to manage exploration and exploitation, they differ in how they quantify uncertainty, handle randomness, and select actions [66, 77]. UCB constructs an upper confidence bound for each action and always selects the action with the highest upper bound [8], whereas TS samples from the posterior distribution over rewards and chooses the action with the highest sampled value [66]. As a result, UCB deterministically selects the same action given the same inputs, while TS's choice varies across runs due to its sampling-based nature [66]. Although UCB is often easier to implement, as it does not require specifying prior or posterior distributions, it can be more brittle to tune (e.g., selecting an appropriate exploration multiplier α) [8, 50, 77]. In contrast, TS is generally more robust to hyperparameter choices and tends to perform better in non-stationary environments [17, 66].

Studies by Chapelle and Li [17] show that TS can perform as well as, or better than, UCB in terms of cumulative regret, defined as the total loss incurred from not consistently selecting the optimal action [17, 66]. This is especially true in scenarios where data is sparse or feedback is delayed [17]. In contrast to UCB, which might over-explore uncertain actions with low potential returns, TS ensures a more balanced and probabilistic form of exploration [66]. TS is well-suited to industry problems such as selecting ads for new or existing users in dynamic environments like online advertising, where user preferences shift frequently and new content is introduced continuously; Chapelle and Li evaluate it empirically on display-advertising data [17]. However, because TS relies on randomized sampling, it can exhibit high variance, which may lead to less stable performance in environments that prioritize short-term accuracy or require highly reliable decision-making [66].

4.6.5 ***Broader Considerations for Exploration and Exploitation.*** While the previous discussion focused on canonical exploration strategies commonly used in industrial recommender and ad systems (e.g., ϵ-Greedy, UCB, Thompson Sampling), it is also important to recognize that additional techniques are often employed to address more complex, high-dimensional, or rapidly evolving environments. For instance, in dynamic ad allocation systems, parameters are often tuned in real-time to balance short-term revenue generation against user experience metrics such as Click Yield, which measures the rate of user interactions per ad impression [99]. By adjusting trade-off parameters in response to shifting engagement patterns, platforms can dynamically prioritize either immediate monetization or sustained user satisfaction. Moreover, alternative approaches such as entropy-regularized policies [58], Bayesian optimization [73], and policy gradient methods like Proximal Policy Optimization (PPO) [71] extend the exploration-exploitation framework in settings where continuous adaptation is critical. Table 11 summarizes how the exploration methods discussed in this subsection compare on their underlying mechanism, strengths, weaknesses, and the ad-policy settings in which each is most appropriate.

Method	How It Works	Pros	Cons	Best Use Cases
Epsilon-Greedy	Selects the best-known action with probability $1-\epsilon$ and explores randomly with probability ϵ.	• Simple to implement • Low computational cost • Easy to understand	• Fixed exploration rate • Might miss better actions • Not adaptive to context	• Simple, static environments • Prioritize computational efficiency
Upper Confidence Bound (UCB)	Scores each action by estimated reward plus an uncertainty bonus; UCB1 uses pull counts while the contextual variant LinUCB uses a linear model over features.	• Systematic exploration • Minimizes long-term regret • Strong theoretical foundations	• May overexplore uncertain actions • Higher computational complexity	• Real-time ad placement • Long-term optimization
Regret Confidence Bounds (RegCB)	Extends UCB by using regression oracles to compute confidence intervals based on contextual features.	• Handles high-dimensional, sparse data • More accurate reward estimates • Well-suited for complex environments	• High computational cost • Difficult to scale	• High-dimensional environments • Needs precise reward prediction
Thompson Sampling (TS)	Uses Bayesian sampling from posterior distributions to estimate optimal actions.	• Adapts quickly to new data • Naturally balances exploration • Effective for cold-start problems	• Can exhibit high variance • Moderate computational complexity	• Cold-start and sparse data • Dynamic, shifting environments

Table 11. Comparison of Exploration-Exploitation Methods in Ad Policy Optimization

5 Evaluating Joint Ad and Organic Ranking Policies

Prior surveys make clear that evaluating RL-based recommenders is not a single-metric problem. Afsar et al. [2] distinguish offline, simulation, and online evaluation environments, while Lin et al. [51] show that many studies still report standard recommendation metrics such as Recall, Hit Ratio, and Normalized Discounted Cumulative Gain (NDCG). However, our literature search did not identify a survey focused on evaluation for systems that jointly optimize organic recommendation and advertising. This creates a gap for monetization-aware ranking: the goal is not to show that every metric improves independently, but to determine whether a policy improves the trade-off among revenue, organic engagement, ad load, advertiser value, and future user tolerance. A policy can therefore be preferable because it increases a weighted utility, because it achieves similar revenue with less engagement degradation, or because it reduces variance and downside risk under user-experience guardrails. This trade-off perspective also changes how offline evidence should be read. Offline ranking protocols can be useful screening tools, but protocols designed around one-step prediction, such as next-item prediction, do not by themselves measure the long-term consequences of a learned policy [23]. The methodology of this section is to use existing RL-recommender surveys as the evaluation baseline, then synthesize primary papers that study advertising, joint ranking, off-policy learning, and industrial rollout. The resulting taxonomy compares objectives and guardrails, pre-deployment screening evidence, and online evidence from deployed systems.

5.1 Evaluation Metrics: Trade-offs and Guardrails

The first part of this evaluation framework specifies which metrics define the trade-off, which metrics diagnose supporting mechanisms, and which metrics act as guardrails. Section 4.2 already introduced the main key performance indicator (KPI) families used in monetization-aware recommendation. We organize these metrics into three roles. *Primary metrics* define the target trade-off, such as weighted utility, revenue, Click Yield, or a platform-specific blend of monetization and engagement. *Secondary metrics* help explain why the primary metric moved, such as CTR, pCTR, dwell time, session duration, or short-view rate. *Guardrail metrics* define outcomes that should not degrade beyond an acceptable threshold, such as return rate, session frequency, ad-load quits, organic engagement, and ad-fatigue indicators [67, 94, 99].

The distinction between short-term proxies and long-term outcomes is central to this hierarchy. Short-horizon metrics such as CTR, pCTR, immediate revenue, dwell time, and short-view rate are useful because they are dense, quickly observed, and sensitive to policy changes. However, the outcomes that ultimately matter for a joint ad and organic ranking policy often unfold over longer horizons: retention, return rate, repeated sessions, ad tolerance, and sustained organic engagement [37, 55, 85]. The same KPI can therefore support different evaluation claims depending on the policy objective. In a constrained ad-allocation system, revenue may be the objective while Click Yield or organic dwell time functions as a protected constraint [94, 99]. In a joint ad and recommendation policy, revenue and engagement may both enter the reward through learned or manually calibrated weights [101, 103]. Evaluation should therefore report not only which metric changed, but also whether the policy improved the intended trade-off: higher weighted utility, less degradation in a protected metric, lower variance, or fewer tail-risk outcomes. This distinction matters because an apparent gain in CTR, pCTR, or revenue can coincide with lower retention, weaker organic engagement, or greater ad fatigue.

5.2 Pre-Deployment Evidence: Offline Replay and Simulation

The second part of this framework concerns evidence gathered before full live deployment. Offline replay and counterfactual policy evaluation are useful because they use logged impressions to estimate how a candidate policy might have performed before that policy is deployed. The central difficulty is that logged data are generated by a behavior or logging policy $\mu(a \mid s)$, whereas the candidate policy $\pi_\theta(a \mid s)$ may choose different ads, placements, or ad-load actions in the same state. Inverse propensity scoring estimates target-policy value by weighting observed outcomes according to how likely the target and logging policies were to select the logged action. Doubly robust estimators combine propensity correction with model-based reward or value estimates [25, 78]. These methods help screen many policy variants cheaply, but they remain sensitive to support: if the logging policy rarely selected a candidate action in a user context, the counterfactual estimate for that action becomes unreliable [19, 49].

Simulator-based testing addresses a different part of the same pre-deployment problem. When live exploration is expensive or risky, simulators allow researchers to test sequential policies under explicit assumptions about user response and state evolution. RecSim provides configurable user-response dynamics for recommender-system experimentation, and SlateQ evaluates slate recommendation through a tractable decomposition under a modeled user-choice process [39, 40]. These tools are valuable for studying repeated exposure, slate interactions, and delayed engagement before deployment. Their limitation is representativeness: a simulator may omit auction dynamics, advertiser budgets, inventory shifts, or changes in user tolerance. For that reason, strong simulated performance should be treated as policy-screening evidence, not as a substitute for online validation [26].

5.3 Online Evidence Across Industrial Systems

The third part of this framework concerns evidence from real traffic. Online A/B tests and staged rollouts remain the strongest evidence that an RL-based ad policy improves production outcomes because they measure the policy under the same auctions, inventories, user populations, and serving constraints that the deployed system faces. Standard experiments are best suited to short-term objective and secondary metrics, including revenue, CTR, Click Yield, dwell time, session length, and short-view rate [9, 37]. Long-term holdout groups play a different role: they estimate whether the policy preserves or improves outcomes that short experiments only proxy, such as retention, return rate, repeated sessions, ad tolerance, and sustained organic engagement [37, 55]. Industrial recommender-system reports also show a common workflow in which candidate policies are first screened offline and then validated online before broader deployment [19, 34].

Table 12 summarizes the evaluation mechanisms that recur across the reviewed systems. The table is organized by the role each mechanism plays in the deployment workflow rather than by paper. This framing makes the evaluation trade-off explicit: offline methods and simulation help narrow the policy search space, while online tests and holdouts validate whether the remaining candidates improve the intended trade-off under real traffic.

Taken together, these practices define a staged evaluation workflow for practitioners. First, define the objective and guardrails using the metric families in Tables 3, 4, and 5. Second, use offline replay, counterfactual estimators, or simulation to remove policies that are unsupported, unsafe, or unlikely to improve the target utility. Third, validate the strongest candidates through staged online experiments with both short-term and long-term guardrails. To provide a concrete perspective on the magnitude of these improvements, Table 13 aggregates reported quantitative results and performance lifts from representative industrial deployments and foundational algorithms. This workflow does not

Measurement mode	Evaluation method	Purpose	Metrics used	Representative systems and sources
Offline	Offline replay / off-policy evaluation	Narrow the search space of candidate policies before live traffic by estimating value under logged impressions	Primary, secondary, and guardrail metrics estimated from logs, such as CTR, pCTR, revenue, dwell time, ad load, organic engagement, Recall, Hit Ratio, NDCG, and learned value estimates	Ad-load balancing via off-policy learning [68]; YouTube off-policy actor-critic [19]; Horizon offline evaluation workflow [34]
Simulation	User-response simulator or slate model	Test sequential effects and stress candidate policies when live exploration is risky or unsupported by logs	The same primary, secondary, and guardrail families measured under modeled user response, including simulated reward, engagement, diversity, Recall, Hit Ratio, NDCG, ad load, and long-term satisfaction	RecSim provides configurable user-response simulation; SlateQ provides a tractable slate-evaluation model [39, 40]
Online	A/B test or staged rollout	Validate whether screened policies improve the intended trade-off under real auctions, inventories, and users	Primary metrics such as revenue, Click Yield, CTR, and weighted utility; guardrails such as organic engagement and ad-fatigue indicators	DEAR / TikTok [101]; joint recommendation and advertising [103]; Alibaba whole-page optimization [99]; LinkedIn feed ads [94]
Online, long horizon	Holdout or delayed-outcome measurement	Measure outcomes that short experiments only proxy, especially retention and future ad tolerance	Return rate, repeated sessions, retention, long-term engagement, ad tolerance, and sustained organic consumption	Long-term holdout and delayed-outcome evaluation in large-scale recommendation systems [37, 55]

Table 12. Evaluation settings, mechanisms, metrics, and representative systems or methodological sources for joint ad and organic ranking policies.

remove the need for live experimentation. It reduces the number of risky policies tested on users and clarifies which trade-offs each evaluation stage can and cannot resolve.

System / Source	Domain / Context	Key Metrics	Reported Quantitative Results
DEAR [101]	Short-video (TikTok)	Accumulated reward	Outperformed hierarchical Deep Q-Network (HDQN) baseline (10.96 vs 10.27) with $p < 0.05$.
Jointly Learning [103]	E-commerce feed	Dwell Time, Ad Revenue	+0.61% Session Dwell Time, +0.83% Session Length, +4.70% Ad Revenue.
Constrained actor-critic (AC) [13]	Short-video (KuaiShou)	WatchTime, Interactions	+0.379% WatchTime, +3.376% Shares, +1.733% Downloads in live A/B tests.
Multi-Objective Bandits [57]	Audio streaming (Spotify)	Clicks, Streams, Generalized Gini Index (GGI)	+6.9% clicks, +11.0% streams, +10.0% GGI over single-objective baselines.
Ad Close Mitigation [75]	Native ads (Yahoo Gemini)	Ad Close Rate, Revenue	>20% reduction in ad close events while limiting revenue decrease to <0.4%.
User Fatigue [67]	Recommender feeds	CTR, Dwell Time	2%–6% relative improvements in CTR and dwell time across different refresh depths.
FeedRec [109]	Recommender feeds	Clicks, Browsing Depth	Outperformed Deep Deterministic Policy Gradient (DDPG) baselines with strong statistical significance ($p < 0.01$).
Budget Bidding [88]	Real-Time Bidding (RTB)	Acquired clicks, R/R^*	Up to 100.92% improvement in R/R^*, +4.3% overall acquired real clicks.
Doubly Robust [25]	Offline Evaluation	RMSE, Bias	10%–20% (average 13.6%) reduction in RMSE compared to standard IPS.

Table 13. Reported quantitative results and performance lifts from representative industrial deployments and foundational algorithms.

6 Conclusions

This review shows that reinforcement learning for ad-supported recommendation is best understood not as a single algorithmic choice, but as a design framework for sequential, monetization-aware control. We organize prior work around the core components of an RL formulation: reward design, action spaces, state representations, policy learning, exploration, and evaluation. This taxonomy is itself a central contribution of the survey. It makes visible that progress in RL-based ad recommendation depends less on applying RL generically, and more on aligning each component of the formulation with the operational constraints of joint ad and organic ranking.

The central difficulty is that joint optimization across ads and organic content is structurally hard. The platform must balance conflicting objectives across users, advertisers, and the recommender system; observe only partial and biased feedback; learn from sparse, delayed, and low-density outcomes; and reason about counterfactual policies using logs generated by earlier behavior policies. These constraints explain why reward design remains the main bottleneck. Useful reward functions and proxy metrics must be observable in the short term, predictive of long-term value, sufficiently dense for learning, and separated enough from their input features to avoid circular optimization. Similarly, richer state and action spaces can represent more granular user, item, slate, and ad-policy decisions, but they also increase variance, reduce support in logged data, and make policy learning more difficult. The design problem is therefore a trade-off between expressiveness and learnability.

Policy learning and exploration must be disciplined for the same reason. In production ad systems, the policy being evaluated is usually not the policy that generated the data, so offline learning and counterfactual estimation are inherently noisy and support-limited. Exploration methods such as bandits are attractive because they offer tractable ways to learn from interaction, but their practical value depends on conservative implementation rather than mathematical simplicity alone. A credible deployment path should begin with simple, auditable policies, increase exploration complexity only when the data justify it, and validate each step against business, advertiser, and user-experience constraints.

For this reason, evaluation is not a final reporting step but a core part of RL system design. Reliable deployment requires a staged workflow: offline estimation to screen candidate policies, simulation tools such as RecSim where they can expose long-horizon dynamics and stress-test assumptions, controlled online experiments to establish causal effects, and longer-term monitoring to detect delayed harms such as user fatigue or advertiser-value degradation. The main lesson from the reviewed literature is therefore that RL can support ad-policy optimization only when its formulation, learning procedure, exploration strategy, and evaluation pipeline are designed together. Future work should focus on source-verified counterfactual evaluation, interpretable state and action abstractions, reward proxies that better predict long-term platform value, and reproducible benchmarks for industrial ad-recommendation settings.

Acknowledgments

The authors thank Adam Obeng, Bee-Chung Chen, and Jay Adams for feedback on industrial practices in monetization-aware recommender systems.

References

[1] Zoë Abrams and Erik Vee. 2007. Personalized ad delivery when ads fatigue: An approximation algorithm. In *International Workshop on Web and Internet Economics*. Springer, 535–540.

[2] M Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–38.

[3] Alekh Agarwal, Daniel Hsu, Satyen Kale, John Langford, Lihong Li, and Robert Schapire. 2014. Taming the monster: A fast and simple algorithm for contextual bandits. In *International conference on machine learning*. PMLR, 1638–1646.

[4] Deepak Agarwal, David Cornell, Bin Huang, Bee-Chung Chen Lee, Ruoxi Tang, Le Zhang, Avery Wolf, Yuchen Shi, Ye Wang, Yin Guo, Srijan Kumar Garimella, Nikita Korolko, and Sandeep Muthukrishnan. 2019. Online parameter selection for web-based services. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. 1645–1653.

[5] Deepak Agarwal, Souvik Ghosh, Kai Wei, and Siyu You. 2014. Budget pacing for targeted online advertisements at LinkedIn. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. doi:10.1145/2623330.2623366

[6] Deepak K Agarwal and Bee-Chung Chen. 2016. *Statistical methods for recommender systems*. Cambridge University Press.

[7] Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. 2020. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning (ICML)*. 104–114.

[8] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine learning* 47 (2002), 235–256.

[9] Joel Barajas, Narayan Bhamidipati, and James G Shanahan. 2022. Online advertising incrementality testing: Practical lessons, paid search and Emerging challenges. In *European Conference on Information Retrieval*. Springer, 575–581.

[10] Alberto Bietti, Alekh Agarwal, and John Langford. 2021. A contextual bandit bake-off. *Journal of Machine Learning Research* 22, 133 (2021), 1–49.

[11] Sébastien Bubeck, Nicolo Cesa-Bianchi, et al. 2012. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning* 5, 1 (2012), 1–122.

[12] Giuseppe Burtini, Jason L. Loeppky, and Ramon Lawrence. 2015. Improving online marketing experiments with drifting multi-armed bandits. In *Proceedings of the 7th International Conference on Agents and Artificial Intelligence (ICAART)*, Vol. 1. SCITEPRESS, 135–146.

[13] Qingpeng Cai, Zhenghai Xue, Chi Zhang, Wanqi Xue, Shuchang Liu, Ruohan Zhan, Xueliang Wang, Tianyou Zuo, Wentao Xie, Dong Zheng, et al. 2023. Two-stage constrained actor-critic for short video recommendation. In *Proceedings of the ACM Web Conference 2023*. 865–875.

[14] Carlos Carrion, Zenan Wang, Harikesh Nair, Xianghong Luo, Yulin Lei, Xiliang Lin, Wenlong Chen, Qiyu Hu, Changping Peng, Yongjun Bao, et al. 2023. Blending advertising with organic content in e-commerce via virtual bids. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 15476–15484. doi:10.1609/aaai.v37i13.26835

[15] Fatih Çelik, Mehmet Safa Çam, and Mehmet Ali Koseoglu. 2023. Ad avoidance in the digital context: A systematic literature review and research agenda. *International Journal of Consumer Studies* 47, 6 (2023), 2071–2105.

[16] Deepayan Chakrabarti, Deepak Agarwal, and Vanja Josifovski. 2008. Contextual advertising by combining relevance with click feedback. In *Proceedings of the 17th international conference on World Wide Web*. 417–426.

[17] Olivier Chapelle and Lihong Li. 2011. An empirical evaluation of thompson sampling. *Advances in neural information processing systems* 24 (2011).

[18] Bee-Chung Chen, Dmitry Pavlov, John F Canny, and Lodewijk Rijckaert. 2009. Large-scale behavioral targeting. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '09)*. 209–218.

[19] Minmin Chen, Can Xu, Vince Gatto, Devanshu Jain, Aviral Kumar, and Ed Chi. 2022. Off-policy actor-critic for recommender systems. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 338–349.

[20] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.

[21] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

[22] Qing Cui, Feng-Shan Bai, Bin Gao, and Tie-Yan Liu. 2015. Global optimization for advertisement selection in sponsored search. *Journal of Computer Science and Technology* 30 (2015), 295–310.

[23] Romain Deffayet, Thibaut Thonet, Jean-Michel Renders, and Maarten de Rijke. 2022. Offline evaluation for reinforcement learning-based recommendation: A critical issue and some alternatives. *ACM SIGIR Forum* 56, 2 (2022).

[24] Christos Dimitrakakis and Ronald Ortner. 2018. Decision making under uncertainty and reinforcement learning. *Book available at http://www. cse. chalmers. se* (2018).

[25] Miroslav Dudík, John Langford, and Lihong Li. 2011. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*. Omnipress, 1097–1104. https://icml.cc/Conferences/2011/papers/554_icmlpaper.pdf

[26] Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468. doi:10.1007/s10994-021-05961-4

[27] Andrew Ellam and Marco Ottaviani. 2003. Overture and Google: Internet pay-per-click (PPC) advertising auctions. *London Business School Case Study CS-03-022* (2003).

[28] Yagil Engel and David Maxwell Chickering. 2008. Incorporating user utility into sponsored-search auctions. In *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 3*. 1565–1568.

[29] Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. 2021. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese* 198, Suppl 27 (2021), 6435–6467.

[30] Daniel C Fain and Jan O Pedersen. 2006. Sponsored search: A brief history. *Bulletin-American Society For Information Science And Technology* 32, 2 (2006), 12.

[31] Dylan Foster, Alekh Agarwal, Miroslav Dudik, Haipeng Luo, and Robert Schapire. 2018. Practical contextual bandits with regression oracles. In *Proceedings of the 35th International Conference on Machine Learning*. PMLR, 1539–1548.

[32] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.

[33] Yuan Gao, Kaiyu Yang, Yuanlong Chen, Min Liu, and Noureddine El Karoui. 2022. Bidding agent design in the LinkedIn ad marketplace. In *Proceedings of the 2022 AdKDD Workshop (KDD)*.

[34] Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. 2018. Horizon: Facebook's open source applied reinforcement learning platform. *arXiv preprint arXiv:1811.00260* (2018). https://research.facebook.com/publications/horizon-facebooks-open-source-applied-reinforcement-learning-platform/

[35] Claire Glanois, Paul Weng, Matthieu Zimmer, Dong Li, Tianpei Yang, Jianye Hao, and Wulong Liu. 2024. A survey on interpretable reinforcement learning. *Machine Learning* (2024), 1–44.

[36] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time personalization using embeddings for search ranking at airbnb. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 311–320.

[37] Henning Hohnhold, Deirdre O'Brien, and Diane Tang. 2015. Focusing on the long-term: It's good for users and business. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1849–1858.

[38] Yu Jeffrey Hu. 2004. Performance-based pricing models in online advertising. *Available at SSRN 501082* (2004).

[39] Eugene Ie, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. RecSim: A configurable simulation platform for recommender systems. *arXiv preprint arXiv:1909.04847* (2019). https://research.google/pubs/recsim-a-configurable-simulation-platform-for-recommender-systems/

[40] Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Tushar Chandra, and Craig Boutilier. 2019. SlateQ: A tractable decomposition for reinforcement learning with recommendation sets. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 2592–2599. doi:10.24963/ijcai.2019/360

[41] Dietmar Jannach and Himan Abdollahpouri. 2023. A survey on multi-objective recommender systems. *Frontiers in Big Data* 6 (2023). doi:10.3389/fdata.2023.1157899

[42] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial intelligence* 101, 1-2 (1998), 99–134.

[43] Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. 1996. Reinforcement learning: A survey. *Journal of artificial intelligence research* 4 (1996), 237–285.

[44] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035

[45] Tanya Kant. 2021. A history of the data-tracked user. *The MIT Press Reader* (2021). https://thereader.mitpress.mit.edu/a-history-of-the-data-tracked-user/

[46] Iordanis Koutsopoulos and Panagiotis Spentzouris. 2016. Native advertisement selection and allocation in social media post feeds. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2016, Riva del Garda, Italy, September 19-23, 2016, Proceedings, Part I 16*. Springer, 588–603.

[47] Volodymyr Kuleshov and Doina Precup. 2014. Algorithms for multi-armed bandit problems. *arXiv preprint arXiv:1402.6028* (2014). https://arxiv.org/abs/1402.6028

[48] Nathan Lambert, Thomas Krendl Gilbert, and Tom Zick. 2023. The history and risks of reinforcement learning and human feedback. *arXiv preprint arXiv:2310.13595* (2023).

[49] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643* (2020).

[50] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*. 661–670.

[51] Yuanguo Lin, Yong Liu, Fan Lin, Lixin Zou, Pengcheng Wu, Wenhua Zeng, Huanhuan Chen, and Chunyan Miao. 2023. A survey on reinforcement learning for recommender systems. *IEEE Transactions on Neural Networks and Learning Systems* (2023).

[52] Zhuoran Liu, Leqi Zou, Xuan Zou, Caihua Wang, Biao Zhang, Da Tang, Bolin Zhu, Yijie Zhu, Peng Wu, Ke Wang, et al. 2022. Monolith: real time recommendation system with collisionless embedding table. *arXiv preprint arXiv:2209.07663* (2022).

[53] Zhongqi Lu and Qiang Yang. 2016. Partially observable Markov decision process for recommender systems. *arXiv preprint arXiv:1608.07793* (2016).

[54] Bogdan Mazoure, Paul Mineiro, Pavithra Srinath, Reza Sharifi Sedeh, Doina Precup, and Adith Swaminathan. 2022. Improving long-term metrics in recommendation systems using short-horizon reinforcement learning. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. https://www.microsoft.com/en-us/research/publication/improving-long-term-metrics-in-recommendation-systems-using-short-horizon-reinforcement-learning/

[55] Thomas M McDonald, Lucas Maystre, Mounia Lalmas, Daniel Russo, and Kamil Ciosek. 2023. Impatient bandits: Optimizing recommendations for the long-term without delay. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1687–1697. doi:10.1145/3580305.3599410

[56] H Brendan McMahan, Gary Holt, David Sculley, Michael Young, Dietmar Ebner, Julian Grady, Lan Nie, Todd Phillips, Eugene Davydov, Daniel Golovin, et al. 2013. Ad click prediction: a view from the trenches. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1222–1230.

[57] Rishabh Mehrotra, Niannan Xue, and Mounia Lalmas. 2020. Bandit based optimization of multiple objectives on a music streaming platform. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*. 3224–3233. doi:10.1145/3394486.3403392

[58] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. PMLR, 1928–1937.

[59] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).

[60] Nielsen. 2017. When it comes to advertising effectiveness, what is key? Nielsen Insights Online Article. https://www.nielsen.com/insights/2017/when-it-comes-to-advertising-effectiveness-what-is-key/ Accessed: 2025-03-27.

[61] Yunzhu Pan, Nian Li, Chen Gao, Jianxin Chang, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. 2023. Learning and optimization of implicit negative feedback for industrial short-video recommender system. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4787–4793.

[62] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[63] Martin L Puterman. 2014. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons.

[64] Herbert Robbins. 1952. Some aspects of the sequential design of experiments. *Bull. Amer. Math. Soc.* 58, 5 (1952), 527–535. doi:10.1090/S0002-9904-1952-09620-8

[65] Stuart J Russell and Peter Norvig. 2016. *Artificial intelligence: a modern approach*. pearson.

[66] Daniel J Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen, et al. 2018. A tutorial on Thompson sampling. *Foundations and Trends® in Machine Learning* 11, 1 (2018), 1–96.

[67] Hitesh Sagtani, Madan Gopal Jhawar, Akshat Gupta, and Rishabh Mehrotra. 2023. Quantifying and leveraging user fatigue for interventions in recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2293–2297.

[68] Hitesh Sagtani, Madan Gopal Jhawar, Rishabh Mehrotra, and Olivier Jeunen. 2024. Ad-load balancing via off-policy learning in a content marketplace. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 586–595.

[69] Koustuv Saha, Yozen Liu, Nicholas Vincent, Farhan Asif Chowdhury, Leonardo Neves, Neil Shah, and Maarten W Bos. 2021. Advertiming matters: Examining user ad consumption for effective ad allocations on social media. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–18.

[70] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*. PMLR, 1889–1897. https://proceedings.mlr.press/v37/schulman15.pdf

[71] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[72] Eric M Schwartz, Eric T Bradlow, and Peter S Fader. 2017. Customer acquisition via display advertising using multi-armed bandit experiments. *Marketing Science* 36, 4 (2017), 500–522.

[73] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. 2015. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE* 104, 1 (2015), 148–175.

[74] Natalia Silberstein, Or Shoham, and Assaf Klein. 2023. Combating ad fatigue via frequency-recency features in online advertising systems. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4822–4828.

[75] Natalia Silberstein, Oren Somekh, Yair Koren, Michal Aharon, Dror Porat, Avi Shahar, and Tingyi Wu. 2020. Ad close mitigation for improved user experience in native advertisements. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 546–554.

[76] Wall Street Journal Staff. 2003. Yahoo agrees to acquire Overture for $1.63 billion. *The Wall Street Journal* (2003). https://www.wsj.com/articles/SB105818965623153600

[77] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement learning: An introduction* (second ed.). The MIT Press. http://incompleteideas.net/book/the-book-2nd.html

[78] Adith Swaminathan and Thorsten Joachims. 2015. Counterfactual risk minimization: Learning from logged bandit feedback. In *International conference on machine learning*. PMLR, 814–823.

[79] Liang Tang, Rómer Rosales, Ajit Singh, and Deepak Agarwal. 2013. Automatic ad format selection via contextual bandits. In *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management (CIKM)*. doi:10.1145/2505515.2514700

[80] Georgios Theocharous, Philip S. Thomas, and Mohammad Ghavamzadeh. 2015. Personalized ad recommendation for life-time value optimization. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI)*. 1807–1813.

[81] Bram van den Akker, Olivier Jeunen, Ying Li, Ben London, Zahra Nazari, and Devesh Parekh. 2024. Practical bandits: An industry perspective. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 1132–1135.

[82] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[83] Vladislav Vorotilov and Ilnur Shugaepov. 2023. Scaling the Instagram explore recommendations system. *Engineering at Meta* (2023). https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/

[84] George A Vouros. 2022. Explainable deep reinforcement learning: state of the art and challenges. *Comput. Surveys* 55, 5 (2022), 1–39.

[85] Yuyan Wang, Mohit Sharma, Can Xu, Sriraj Badam, Qian Sun, Lee Richardson, Lisa Chung, Ed H Chi, and Minmin Chen. 2022. Surrogate for long-term user experience in recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 4100–4109.

[86] Ying Wen, Yao Mu, and Martin Ester. 2019. Learning multi-objective rewards and user utility function in recommender systems. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization, 3719–3725. doi:10.24963/ijcai.2019/517

[87] Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3-4 (1992), 229–256. doi:10.1007/BF00992696

[88] Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. 2018. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. 1443–1451.

[89] Qingyun Wu, Hongning Wang, Liangjie Hong, and Yue Shi. 2017. Returning is believing: Optimizing long-term user engagement in recommender systems. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1927–1936.

[90] Siqi Wu, Marian-Andrei Rizoiu, and Lexing Xie. 2018. Beyond views: Measuring and predicting engagement in online videos. In *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 12.

[91] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.

[92] Ruiyang Xu, Jalaj Bhandari, Dmytro Korenkevych, Fan Liu, Yuchen He, Alex Nikulkov, and Zheqing Zhu. 2023. Optimizing long-term value for auction-based recommender systems via on-policy reinforcement learning. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 955–962. doi:10.1145/3604915.3608854

[93] Jun Yan, Ning Liu, Gang Wang, Wen Zhang, Yun Jiang, and Zheng Chen. 2009. How much can behavioral targeting help online advertising?. In *Proceedings of the 18th international conference on World wide web*. 261–270.

[94] Jinyun Yan, Zhiyuan Xu, Birjodh Tiwana, and Shaunak Chatterjee. 2020. Ads allocation in feed via constrained optimization. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3386–3394.

[95] Congrui Yi, David Zumwalt, Zijian Ni, and Shreya Chakrabarti. 2023. Progressive horizon learning: Adaptive long term optimization for personalized recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 940–946.

[96] Xing Yi, Liangjie Hong, Erheng Zhong, Nanthan Nan Liu, and Suju Rajan. 2014. Beyond clicks: dwell time for personalization. In *Proceedings of the 8th ACM Conference on Recommender systems*. 113–120.

[97] Bowen Yuan, Yaxu Liu, Jui-Yang Hsia, Zhenhua Dong, and Chih-Jen Lin. 2020. Unbiased Ad click prediction for position-aware advertising systems. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 368–377.

[98] Wei Zhang, Dai Li, Chen Liang, Fang Zhou, Zhongke Zhang, Xuewei Wang, Ru Li, Yi Zhou, Yaning Huang, Dong Liang, et al. 2024. Scaling user modeling: Large-scale online user representations for ads personalization in Meta. In *Companion Proceedings of the ACM Web Conference 2024*. 47–55.

[99] Weiru Zhang, Chao Wei, Xiaonan Meng, Yi Hu, and Hao Wang. 2018. The whole-page optimization via dynamic ad allocation. In *Companion Proceedings of the The Web Conference 2018*. 1407–1411.

[100] Jun Zhao, Guang Qiu, Ziyu Guan, Wei Zhao, and Xiaofei He. 2018. Deep reinforcement learning for sponsored search real-time bidding. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1021–1030.

[101] Xiangyu Zhao, Changsheng Gu, Haoshenglun Zhang, Xiwang Yang, Xiaobing Liu, Jiliang Tang, and Hui Liu. 2021. Dear: Deep reinforcement learning for online advertising impression in recommender systems. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 750–758.

[102] Xiangyu Zhao, Liang Zhang, Zhuoye Ding, Long Xia, Jiliang Tang, and Dawei Yin. 2018. Recommendations with negative feedback via pairwise deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1040–1048.

[103] Xiangyu Zhao, Xudong Zheng, Xiwang Yang, Xiaobing Liu, and Jiliang Tang. 2020. Jointly learning to recommend and advertise. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, New York, NY, USA, 3319–3327. doi:10.1145/3394486.3403233

[104] Yu Zhao, Fang Liu, Yuan Yuan, and Yifan Dang. 2026. A survey of retrieval algorithms in ad and content recommendation systems. *International Journal of Electrical and Computer Engineering (IJECE)* 16, 3 (2026), 1518–1530. doi:10.11591/ijece.v16i3.pp1518-1530

[105] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI)*. 5941–5948. doi:10.1609/aaai.v33i01.33015941

[106] Guorui Zhou, Chengru Song, Xiaoqiang Zhu, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '18)*. 1059–1068. doi:10.1145/3219819.3219823

[107] Li Zhou and Emma Brunskill. 2016. Latent contextual bandits and their application to personalized recommendations for new users. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*. 3646–3653. https://www.ijcai.org/Proceedings/16/Papers/513.pdf

[108] Jie Zhu, Fengge Wu, and Junsuo Zhao. 2021. An overview of the action space for deep reinforcement learning. In *Proceedings of the 2021 4th International Conference on Algorithms, Computing and Artificial Intelligence*. 1–10.

[109] Lixin Zou, Long Xia, Zhuoye Ding, Jiaxing Song, Weidong Liu, and Dawei Yin. 2019. Reinforcement learning to optimize long-term user engagement in recommender systems. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2810–2818.